



**JOÃO AUGUSTO DIAS NETO**

**ANÁLISE GEOESPACIAL E APRENDIZADO DE MÁQUINA  
NA PRECIFICAÇÃO DO AIRBNB**

**LAVRAS – MG**

**2025**

**JOÃO AUGUSTO DIAS NETO**

**ANÁLISE GEOESPACIAL E APRENDIZADO DE MÁQUINA NA PRECIFICAÇÃO  
DO AIRBNB**

Trabalho de conclusão de curso apresentada à  
Universidade Federal de Lavras, como parte das  
exigências do curso de Bacharel em Sistemas  
de Informações, para a obtenção do título de  
Bacharel.

Profa. Dra. Elaine Cecília Gatto  
Orientadora

**LAVRAS – MG**

**2025**

**JOÃO AUGUSTO DIAS NETO**

**ANÁLISE GEOESPACIAL E APRENDIZADO DE MÁQUINA NA PRECIFICAÇÃO  
DO AIRBNB**

Trabalho de conclusão de curso apresentada à  
Universidade Federal de Lavras, como parte das  
exigências do curso de Bacharel em Sistemas  
de Informações, para a obtenção do título de  
Bacharel.

APROVADA em XXXXXX.

XXX XXX  
XXXX XXX

Profa. Dra. Elaine Cecília Gatto  
Orientadora

**LAVRAS – MG  
2025**

*Dedico este trabalho à minha família e amigos e, especialmente, aos meus pais, Cláudio e Rozilene, que, sob muito sol, me fizeram chegar até aqui, na sombra.*

## **AGRADECIMENTOS**

À minha família e amigos, pelo amor e apoio incondicional. À Profa. Dra. Elaine Cecília Gatto, pela orientação e inspiração.

*“Só se pode alcançar um grande êxito quando nos mantemos fiéis a nós mesmos.”  
(Friedrich Nietzsche)*

## RESUMO

Este trabalho apresenta uma investigação detalhada sobre a predição de preços de acomodações do Airbnb localizadas em Sydney e região. O estudo emprega técnicas avançadas de aprendizado de máquina sobre um conjunto de dados públicos de listagens. O projeto focou primariamente na coleta e no pré-processamento rigoroso desses dados, bem como na subsequente avaliação de três modelos preditivos, LightGBM, Random Forest e XGBoost, concebidos para resolver a problemática abordada. Adicionalmente, foi gerado um mapa de calor interativo, que confirmou visualmente a significância preditiva da localidade na formação dos preços. O estudo buscou capacitar os algoritmos a compreenderem integralmente o valor da posição geográfica para a precisão da precificação. Concluiu-se que o algoritmo LightGBM foi o de desempenho superior, demonstrando o melhor equilíbrio entre poder preditivo e eficiência computacional, e alcançando resultados satisfatórios em todas as métricas de avaliação. Dessa forma, esta pesquisa contribui para a literatura existente ao validar a importância da localização em modelos robustos de precificação e recomenda a utilização do modelo LightGBM para futuras aplicações mercadológicas, devido ao seu desempenho superior e baixo custo computacional.

**Palavras-chave:** Airbnb; Aprendizado de Máquina; Predição de Preços; Análise Geoespacial; Ciência de Dados.

## ABSTRACT

This work presents a detailed investigation into the price prediction of Airbnb accommodations located in Sydney and its surrounding region. The study employs advanced machine learning techniques on a publicly available dataset of listings. The project primarily focused on the collection, rigorous preprocessing of these data, and the subsequent evaluation of three predictive models, LightGBM, Random Forest and XGBoost, designed to resolve the addressed problem. Furthermore, an interactive heatmap was generated, visually confirming the predictive significance of location in price formation. The study sought to empower the algorithms to fully grasp the value of geographical position for pricing accuracy. It was concluded that the LightGBM algorithm was the superior performer, demonstrating the best balance between predictive power and computational efficiency, and achieving satisfactory results across all evaluation metrics. Thus, this research contributes to the existing literature by validating the importance of location in robust pricing models and recommends the utilization of the LightGBM model for future market applications due to its superior performance and low computational overhead.

**Keywords:** Airbnb; Machine Learning; Price Prediction; Geospatial Analysis; Data Science.



## LISTA DE FIGURAS

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Figura 4.1 – Histograma do atributo de saída . . . . .               | 36 |
| Figura 4.2 – Diagrama de caixa do atributo de saída . . . . .        | 37 |
| Figura 4.3 – Q-Q Plot do atributo de saída . . . . .                 | 37 |
| Figura 4.4 – Mapa de calor de Correlação de Pearson . . . . .        | 38 |
| Figura 4.5 – Ranking de comodidades: Top 20 . . . . .                | 39 |
| Figura 4.6 – Distribuição geográfica dos imóveis por preço . . . . . | 39 |
| Figura 4.7 – Diagrama de distância crítica . . . . .                 | 46 |
| Figura 4.8 – Resíduos vs. Valores Observados . . . . .               | 47 |
| Figura 4.9 – Resíduos vs. Predições . . . . .                        | 48 |

## LISTA DE TABELAS

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Tabela 4.1 – Dicionário de dados das acomodações do Airbnb . . . . .                   | 31 |
| Tabela 4.2 – Atributos com dados faltantes (Percentual > 1%) . . . . .                 | 35 |
| Tabela 4.3 – Resumo Estatístico do atributo de saída . . . . .                         | 35 |
| Tabela 4.4 – Resultados do Coeficiente de Determinação do Grid Search . . . . .        | 42 |
| Tabela 4.5 – Resultados de desempenho da avaliação dos modelos (D.P.: desvio padrão) . | 43 |
| Tabela 4.6 – Resultados de Tempo de Treino, Teste e Busca em Grade . . . . .           | 44 |

## SUMÁRIO

|            |                                                                   |           |
|------------|-------------------------------------------------------------------|-----------|
| <b>1</b>   | <b>INTRODUÇÃO . . . . .</b>                                       | <b>10</b> |
| <b>2</b>   | <b>APRENDIZADO DE MÁQUINA . . . . .</b>                           | <b>11</b> |
| <b>2.1</b> | <b>Aprendizado Supervisionado . . . . .</b>                       | <b>11</b> |
| <b>2.2</b> | <b>Fluxo Metodológico do Aprendizado Supervisionado . . . . .</b> | <b>12</b> |
| <b>2.3</b> | <b>Regressão . . . . .</b>                                        | <b>18</b> |
| <b>2.4</b> | <b>Métricas de Avaliação . . . . .</b>                            | <b>22</b> |
| <b>2.5</b> | <b>Testes Estatísticos . . . . .</b>                              | <b>24</b> |
| <b>3</b>   | <b>METODOLOGIA . . . . .</b>                                      | <b>28</b> |
| <b>3.1</b> | <b>Conjunto de Dados . . . . .</b>                                | <b>28</b> |
| <b>3.2</b> | <b>Ambiente de Implementação . . . . .</b>                        | <b>28</b> |
| <b>3.3</b> | <b>Pipeline . . . . .</b>                                         | <b>28</b> |
| <b>4</b>   | <b>RESULTADOS E DISCUSSÃO . . . . .</b>                           | <b>31</b> |
| <b>4.1</b> | <b>Pré-processamento dos dados . . . . .</b>                      | <b>34</b> |
| <b>5</b>   | <b>CONCLUSÃO . . . . .</b>                                        | <b>49</b> |
|            | <b>REFERÊNCIAS . . . . .</b>                                      | <b>50</b> |

## 1 INTRODUÇÃO

A economia compartilhada surgiu como um modelo econômico importante para redefinir a relação entre consumo e mercado, especialmente na indústria da hospitalidade. Nesse contexto, o Airbnb surge como um caso de sucesso da aplicação desse modelo no ramo da hotelaria. A plataforma, fundada em 2007, disponibiliza propriedades para locação de curta duração, processo mediado pelos proprietários dos imóveis (AIRBNB, 2025).

Segundo Huang (2025) e Zeng, Huo e Zhang (2025), a ascensão da plataforma no mercado global, como alternativa mais acessível e flexível quando comparada à hotelaria tradicional, beneficia particularmente consumidores de baixa renda, o que potencializa de maneira ainda mais relevante seu domínio na indústria.

Contudo, o desconhecimento dos usuários sobre a melhor maneira de precificar uma acomodação, tanto para o inquilino quanto para o anfitrião que a disponibiliza, cria a oportunidade de aplicar ferramentas analíticas que otimizem a gestão de receitas da plataforma. Dessa forma, a aplicação de modelos de Aprendizado de Máquina (AM ou em inglês Machine Learning - ML), capazes de prever o valor de uma listagem, surge como uma estratégia interessante ao gerar insights aos stakeholders e proporcionar aos consumidores uma redução da incerteza inerente a esse mercado emergente.

Este estudo busca compreender a distinção entre modelos de regressão, em termos de desempenho computacional e capacidade preditiva, a fim de determinar o algoritmo mais adequado para ser aplicado em um cenário de produção pela plataforma.

O estudo também realiza uma análise geoespacial para entender como a localização do imóvel se relaciona com o seu preço. Essa análise visa compreender a ligação entre preço e localidade e, especialmente, treinar os modelos para incorporar a importância dessa relação. Essa relação deve ser considerada tanto quanto atributos físicos da acomodação, como comodidades oferecidas e número de quartos, conforme Akalın e Alptekin (2024).

Portanto, o trabalho pode ser relevante tanto para inquilinos quanto para anfitriões da plataforma, pois terão uma visão mais clara e justa sobre a precificação de imóveis. É igualmente significativo para a empresa, que será capaz de implementar uma ferramenta poderosa e eficiente, potencializando e facilitando o trabalho dos interessados.

## 2 APRENDIZADO DE MÁQUINA

O termo Aprendizado de Máquina, introduzido pela primeira vez por Samuel (1959), caracteriza-se pela criação de algoritmos que capacitam as máquinas a aprender a realizar tarefas em um conjunto de dados específico, sem a necessidade de instruções explícitas para a execução. Assim, este subcampo da Inteligência Artificial estuda como as máquinas podem emular comportamentos e habilidades humanas. Essas máquinas tornam-se capazes de extrair informações proveitosas de dados, realizar tarefas e identificar padrões, aprendendo e se adaptando de forma mais rápida e eficiente quando comparadas a um ser humano.

O Aprendizado de Máquina é prioritariamente classificado em três categorias: Aprendizado Supervisionado, que aprende com dados rotulados, Aprendizado Não Supervisionado, que aprende com dados não rotulados e Aprendizado por Reforço, que aprende com tentativa e erro. A seção subsequente discorrerá sobre o Aprendizado Supervisionado, categoria na qual se enquadra o problema que este trabalho busca solucionar (REZAZADEH, 2025).

### 2.1 Aprendizado Supervisionado

O Aprendizado Supervisionado se destaca como uma das principais abordagens utilizadas no Aprendizado de Máquina, segundo Rezazadeh (2025). Essa categoria busca treinar modelos para aprenderem como utilizar dados de entrada para encontrar uma saída alvo correspondente. A partir de um conjunto de dados, busca-se a maneira mais eficaz de transformar os dados nas saídas desejadas. Posteriormente, os algoritmos são avaliados através de métricas de desempenho. O processo utiliza dados já rotulados, isto é, dados que já possuem algum tipo de informação fornecida por humanos, a fim de adicionar contexto ou significado adicional. O processo também pode adicionar novos dados rotulados. Desta forma, como analisado por Rezazadeh (2025), os algoritmos de Aprendizado Supervisionado operam sobre esses dados, examinando os mesmos minuciosamente e determinando rótulos apropriados a dados novos não rotulados, baseando-se em padrões e observações feitas no processo de aprendizado.

No que diz respeito às aplicações práticas, o Aprendizado Supervisionado tem suas tarefas categorizadas em duas principais classificações: Classificação e Regressão. A primeira é empregada em tarefas onde se busca uma saída discreta ou categórica, ou seja, busca-se pro-

priamente classificar algo, segundo Rezazadeh (2025). Por outro lado, a Regressão é utilizada quando se busca uma saída contínua, isto é, para prever um valor numérico em um determinado intervalo, como o preço de uma listagem em um conjunto de listagens de acomodações do Airbnb, o contexto deste trabalho.

## 2.2 Fluxo Metodológico do Aprendizado Supervisionado

No que tange à aplicação de um projeto de Aprendizado Supervisionado, é necessário seguir um fluxo metodológico de implementação com etapas bem definidas. Primeiramente, é necessário definir o problema a ser tratado, de forma a classificá-lo como Classificação ou Regressão. Posteriormente, deve ser realizada a coleta de dados, com o objetivo de reunir um conjunto de dados rotulados que sejam representativos. Em seguida, deve-se executar um pré-processamento desses dados, o que engloba todo o processo de limpeza, normalização, codificação e, caso necessário, engenharia de atributos, na qual novos atributos são gerados a partir dos já existentes. Por fim, realiza-se a divisão do conjunto de dados em subconjuntos, sendo estes: treinamento, validação e teste (BDAIR, 2022).

### Análise de Normalidade

De acordo com Das (2016), métodos estatísticos como a Regressão se baseiam na premissa fundamental de que a observação segue uma distribuição normal (Gaussiana), assumindo-se que as populações amostradas também são normalmente distribuídas. No entanto, a realidade é que as distribuições não normais são altamente frequentes na prática. Por essa razão, torna-se imperativo que a premissa de normalidade seja verificada antes da aplicação de métodos inferenciais.

Uma das técnicas para analisar a normalidade de dados, apresentada por Das (2016), é denominada plot quantil-quantil (Q-Q). Esta técnica compara os quantis de uma distribuição teórica padronizada de uma família de distribuições específicas. Assim, um plot Q-Q normal nos possibilita modelar plotando os quantis de uma distribuição versus os quantis de uma distribuição normal. No momento em que as distribuições se encontram, os pontos plotados se alinham com a linha  $y = x$ . Se o gráfico mostrar um formato de curva com a inclinação su-

bindo da esquerda pra direita, existe um indício de distribuição dos dados enviesada pra direita (assimetria positiva). O cenário contrário sinaliza uma assimetria negativa.

Portanto, os quantis teóricos são calculados a partir de uma distribuição padrão dentro da família de distribuições especificada. Para a geração do Q-Q plot para o atributo de saída, define-se que os quantis do mesmo sejam comparados com os quantis de uma distribuição normal padrão. Com isso, caso os pontos plotados estiverem alinhados à linha reta, existe uma distribuição normal.

Outra técnica é o teste de Shapiro-Wilk, proposto por [Shapiro e Wilk \(1965\)](#) baseado na correlação entre as observações fornecidas e suas pontuações normais associadas. O teste retorna um valor  $W$  entre 0 e 1, onde valores próximos a 1 indicam normalidade, e o cenário inverso indica anormalidade na distribuição. A fórmula da estatística de teste é dada na Equação 2.1.

$$W = \frac{\left( \sum_{i=1}^n a_i y_{(i)} \right)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (2.1)$$

Onde  $y_{(i)}$  é a  $i$ -ésima estatística de ordem (o dado ordenado) e  $a_i$  é o  $i$ -ésimo valor esperado das estatísticas de ordem normalizadas. Para observações independentes e identicamente distribuídas, os valores de  $a_i$  podem ser obtidos na tabela apresentada por [Shapiro e Wilk \(1965\)](#) para tamanhos de amostra de até 50. Em complemento, [Cowles e Davis \(1982\)](#) estabeleceu o nível de significância  $\alpha = 0.05$  como padrão universal. Este valor define o risco máximo de cometer um Erro Tipo I (falso positivo) que se está disposto a tolerar.

Definir  $\alpha = 0.05$  significa aceitar apenas uma chance de 5% de rejeitar a hipótese nula de que os dados são normais quando, na verdade, ela é verdadeira. O  $p$ -valor, que é definido a partir da estatística  $W$  (por meio de tabelas de distribuição apresentadas por [Shapiro e Wilk \(1965\)](#) que consultam  $W$  e o tamanho da amostra  $n$ ), é então comparado com  $\alpha$ . Caso o  $p$ -valor seja menor que 0.05, a probabilidade de a não-normalidade observada ser devida ao acaso é inferior a 5%, o que permite afirmar que é estatisticamente seguro rejeitar a hipótese nula. Cabe ressaltar que a limitação prática do número de observações a serem testadas para o Shapiro-Wilk é de 5000 amostras.

## Detecção e tratamento de anomalias

Em conformidade com o proposto por [Montgomery \(2024\)](#), a detecção de anomalias (outliers) é um passo fundamental na análise de dados e no aprendizado de máquina. Esse procedimento consiste em identificar dados que se desviam significativamente do comportamento padrão do conjunto. Sua importância reside no fato de que essas anomalias podem tanto indicar ruídos ou erros de medição quanto sinalizar eventos raros e informações relevantes (como um valor extremo). Portanto, detectar e gerenciar essas discrepâncias é importante para garantir a acurácia e a confiabilidade dos modelos.

Uma das estratégias para a detecção de anomalias é o diagrama de caixa juntamente com o Intervalo Interquartil (IQR). Estas ferramentas permitem a visualização da distribuição dos dados usando quartis, identificando as anomalias por meio do IQR. As anormalidades são representadas pelos pontos de dados que se situam além de 1,5 vezes o IQR (Intervalo Interquartil) acima do terceiro quartil ou abaixo do primeiro quartil ([MONTGOMERY, 2024](#); [DAS, 2016](#)).

O diagrama de caixa é construído da seguinte forma: tenta-se plotar os dados em uma caixa cujo ponto central indica a mediana da amostra, o topo indica o terceiro quartil (Q3) e a base revela o primeiro quartil (Q1). O “bigode” superior se estende até o valor adjacente, isto é, o valor dado mais alto dentro do limite superior, igual a:  $Q3 + 1,5 \times IQR$  (onde o IQR é calculado por:  $IQR = Q3 - Q1$ ). A visualização do diagrama de caixa oferece uma indicação clara sobre a simetria do conjunto de dados observados. De forma análoga, o “bigode” inferior se estende até o valor adjacente, este representante do valor mais baixo dentro do limite inferior, igual a:  $Q1 - 1,5 \times IQR$  ([DAS, 2016](#)).

Outra estratégia é a geração de um histograma, onde os valores observados são plotados em relação a sua frequência, de forma que é possível estimar visualmente se a distribuição tem formato de sino ou não. Dados que apresentam uma curvatura em formato de sino são ditos como tendo uma distribuição normal, enquanto o cenário contrário indica assimetria e anormalidade. Simultaneamente, é possível encontrar estimativas sobre lacunas de dados e também a presença de anomalias ([DAS, 2016](#)).



A Winsorização, em conformidade com o proposto por Abuzaid e Alkronz (2024), é um método de substituição do valor de anomalias por uma medida estatística adequada para o contexto. Para o presente projeto, a técnica escolhida foi a de ' piso' e 'teto' baseados em quartis. Nessa abordagem, as anomalias máximas são substituídas pelo limite superior ao IQR (teto), e as anomalias mínimas são substituídas pelo limite inferior ao IQR (piso).

### Correlação de Pearson

A correlação de Pearson, introduzida por Pearson (1909), é dada pela Equação 2.2 e busca medir a relação linear entre dois atributos, isto é, o quanto um depende do outro.

$$r = \frac{n(\sum XY) - (\sum X)(\sum Y)}{\sqrt{[n\sum X^2 - (\sum X)^2][n\sum Y^2 - (\sum Y)^2]}} \quad (2.2)$$

Onde  $r$  é o coeficiente de correlação,  $n$  é o número de observações,  $X$  é o primeiro atributo do contexto e  $Y$  é o segundo atributo do contexto. Os resultados são interpretados da seguinte forma:  $r = 0$  indica correlação positiva (ambos crescem juntos);  $r = 1$  indica correlação negativa (um cresce, o outro diminui);  $r = 0$  indica que os atributos não dependem um do outro.

### Colinearidade e Multicolinearidade

Segundo Shrestha (2020), a colinearidade indica que dois atributos são combinações lineares quase perfeitas um do outro, isto é, ambos apresentam informações quase idênticas. Desta forma, a multicolinearidade ocorre quando o modelo de regressão inclui vários atributos (três ou mais) significativamente correlacionados não somente com o atributo de saída, mas também entre si.

Ao lidar com um conjunto de dados com multicolinearidade, os resultados do modelo podem não ser confiáveis. Desse modo, o principal problema da multicolinearidade para modelos de regressão reside nos erros padrão irregulares e viesados e na interpretação impraticável dos coeficientes de regressão (SHRESTHA, 2020).

O algoritmo apresenta dificuldade em discernir o impacto isolado de cada atributo na predição final, devido à redundância da informação contida entre eles. Portanto, é necessário que se apliquem técnicas de detecção e tratamento de multicolinearidade (SHRESTHA, 2020).

Uma das técnicas para encontrar multicolinearidade em um conjunto de dados é o próprio coeficiente de Pearson. Caso o valor absoluto do coeficiente de correlação de Pearson esteja próximo de 1, é possível que exista colinearidade (SHRESTHA, 2020).

Outra técnica utilizada para detectar a multicolinearidade é conhecida como Variance Influence Factor (VIF) (Em português: Fator de Inflação da Variância). Essa medida é usada para medir o quanto a variância do coeficiente de regressão estimada é inflada se os atributos de entrada estiverem correlacionados. A equação do VIF se da pela Equação 2.3 (SHRESTHA, 2020).

$$VIF = \frac{1}{1 - R^2} = \frac{1}{\text{Tolerância}} \quad (2.3)$$

Onde a tolerância é simplesmente o inverso do VIF, e o  $R^2$  representa o coeficiente de determinação, a ser apresentado em seções subsequentes. Quanto menor a tolerância, maior a probabilidade de multicolinearidade entre os atributos. Portanto, os resultados são interpretados da seguinte maneira:  $VIF = 1$  -> atributos não estão correlacionados;  $1 < VIF < 5$  -> moderadamente correlacionados;  $VIF > 10$  -> os coeficientes de regressão são estimados de forma fraca na presença de multicolinearidade (SHRESTHA, 2020).

### **Codificação de Atributos**

O tratamento adequado de atributos categóricos é fundamental para a performance, capacidade de interpretação e generalização de algoritmos baseados em ensemble como Random Forest, XGBoost e LightGBM, segundo Olusegun (2025). Uma vez que esses modelos dependem de atributos exclusivamente numéricos para realizar as previsões, é necessário que atributos categóricos sejam convertidos em números.

Modelos baseados em árvores de decisão dividem os dados com base nos atributos de entrada. Assim, caso os atributos categóricos sejam mal codificados, é possível que divisões abaixo do ideal (subótimas) sejam geradas. Portanto, é necessário escolher a estratégia ideal de conversão, sendo necessário avaliar o contexto do problema e o algoritmo selecionado. Duas estratégias comuns para essa codificação serão abordadas a seguir (OLUSEGUN, 2025).

A codificação one-hot (One-Hot Encoding) é uma técnica onde cada categoria é convertida em vetores binários, isto é, cada categoria única é transformada em uma nova coluna. Embora esse procedimento seja adequado para modelos baseados em árvore, o mesmo pode causar esparsidade em recursos de alta cardinalidade, gerando um contingente massivo de novas colunas, o que eleva de maneira significativa o custo computacional de etapas seguintes. Outra estratégia é a codificação de rótulos (Label Encoding), que atribui rótulos numéricos às categorias (Ex.: Vermelho -> 1; Azul -> 2; Amarelo -> 3), substituindo o valor categórico pelo rótulo atribuído. Assim, é dispensada a necessidade de se criar novas colunas. Entretanto, essa abordagem pode introduzir relações ordinais (de ordem) onde elas não existem (OLUSEGUN, 2025).

### **Validação Cruzada $K$ –Fold**

A técnica de  $K$ –Fold Cross-Validation, ou Validação Cruzada  $K$ –Fold, é utilizada para validar o desempenho dos modelos. Esse procedimento consiste em dividir um conjunto de dados aleatoriamente em  $k$  subconjuntos (folds) disjuntos e de mesmo tamanho. Em seguida, cada subconjunto é utilizado sequencialmente para testar o modelo induzido pelos demais  $k - 1$  subconjuntos. Por fim, o desempenho do algoritmo é avaliado pela média das  $k$  acurácias resultantes desse processo de validação cruzada  $k$ –fold. (WONG, 2015) (DEMIRTRK; MINTEMUR; ARSLAN, 2025)

Esse processo é importante para evitar problemas como o sobreajuste, no qual o modelo se ajusta bem a um determinado conjunto de dados, mas não é capaz de apresentar desempenho satisfatório com novos dados, e o subajuste, no qual o modelo é incapaz de capturar padrões no conjunto de treinamento e, desta forma, desempenha mal no conjunto de teste (BDAIR, 2022) (WONG, 2015) (DEMIRTRK; MINTEMUR; ARSLAN, 2025).

### **Ajuste de Hiperparâmetros**

Uma vez que os dados estão prontos para uso, é necessário realizar o processo de ajuste de hiperparâmetros, no qual é realizada uma busca da melhor combinação de parâmetros para treinar o modelo, com base no conjunto de treinamento formado anteriormente. Com isso, é

possível potencializar a performance do modelo, utilizando os melhores parâmetros encontrados para a tarefa a ser realizada (BDAIR, 2022). Esse processo de busca pode ser realizado de diferentes formas, sendo a mais usual delas a busca em grade (grid search), combinada à validação cruzada. Como foi demonstrado por Liashchynskiy e Liashchynskiy (2019), esse método faz uma varredura completa em um dado intervalo de hiperparâmetros, testando todas as combinações e encontrando a mais adequada, de acordo com uma métrica de avaliação apropriada para o contexto do modelo. Assim, a performance de cada conjunto de hiperparâmetros é então submetida à avaliação, utilizando a métrica apropriada para o tipo de tarefa. Especialmente em casos de regressão,  $R^2$  e  $RMSE$  são usualmente utilizados (LIASHCHYNSKYI; LIASHCHYNSKYI, 2019).

### **Avaliação do Modelo**

Uma vez encontrado o melhor conjunto de hiperparâmetros, é realizado o treinamento do modelo, no qual o mesmo aprende, utilizando os atributos de entrada, a encontrar o valor do atributo de saída. Dessa forma, ao fim, o modelo é testado com um conjunto de dados não vistos, encontrados no conjunto de teste, e avalia-se a capacidade do modelo de realizar a tarefa determinada, isto é, o quão assertivo foi o algoritmo na tarefa proposta (BDAIR, 2022).

O processo culmina com o Teste final em um conjunto de dados não visto e o desempenho é avaliado. É necessário realizar um acompanhamento contínuo do desempenho e garantir o retreinamento conforme as mudanças nos dados. É válido salientar que o processo de treinamento, teste e validação de modelos de aprendizado de máquina visa preparar o modelo para generalizar, de forma que este seja capaz de apresentar desempenho satisfatório mesmo em conjuntos distintos de dados (BDAIR, 2022).

## **2.3 Regressão**

O conceito de Regressão e suas tarefas, introduzidos à comunidade acadêmica por Galton (1886), para o contexto do aprendizado de máquina, caracterizam-se pela busca por prever um valor de saída contínuo (KERN; KLAUSCH; KREUTER, 2019). Por outro lado, as tarefas de Classificação visam categorizar dados, atribuindo-lhes rótulos. Sendo assim, a natureza de

problemas de regressão é variada, abrangendo todo tipo de abordagem em que se busca prever o valor de algo.

Embora exista uma vasta gama de métodos de aprendizado supervisionado a serem utilizados em uma tarefa de regressão, abordagens baseadas em árvores podem ser particularmente interessantes para o contexto do projeto. Esses algoritmos conseguem lidar com dados diversos sem a necessidade de um pré-processamento exaustivo, além de conseguirem pré-selecionar atributos adequados para a predição através da própria construção da árvore. No entanto, os modelos podem se diferenciar na forma como essas árvores de decisão são construídas, afetando diretamente a maneira como cada modelo atua na tarefa. As nuances de cada modelo a ser utilizado no projeto serão abordadas a seguir (KERN; KLAUSCH; KREUTER, 2019).

## Random Forest

A Random Forest (Floresta Aleatória), inicialmente proposta por Breiman (2001), é caracterizada como um modelo de ensemble<sup>1</sup>, e é um algoritmo usualmente utilizado em tarefas de aprendizado supervisionado. Segundo Kern, Klausch e Kreuter (2019), o modelo de árvores de regressão (árvores de decisão para regressão) baseia-se na construção de diversas árvores de decisão preditivas, onde cada árvore trabalha sobre um conjunto aleatório de dados e utilizando um conjunto aleatório de parâmetros, realizando a predição do atributo de saída. O modelo combina as predições de todas as árvores formadas e gera uma predição final através da média desses resultados. Esse processo é também conhecido como Bagging (Bootstrap Aggregating que em português é : Agregação Bootstrap ).<sup>2</sup>

## Boosting

Enquanto em modelos de ensemble como o Random Forest cada árvore cresce de maneira independente da outra, a técnica conhecida como Boosting, inicialmente proposta por Freund e Schapire (1996) (com o AdaBoost, o primeiro algoritmo de boosting prático), concen-

<sup>1</sup> Ensemble é uma técnica onde múltiplas instâncias de modelos preditivos são combinadas, em prol de gerar uma única predição otimizada

<sup>2</sup> Bagging, ou Bootstrap Aggregating, é uma técnica que consiste em treinar múltiplos modelos da mesma natureza em diferentes amostras do conjunto original de dados

tra a família de modelos que focam em aprendizado contínuo. Nesse contexto, as árvores são geradas de maneira sequencial, na qual uma depende dos resultados da anterior para a sua criação. A seguir, o gradient boosting será apresentado (KERN; KLAUSCH; KREUTER, 2019).

## Gradient Boosting

O gradient boosting, introduzido por Friedman (2001), busca encontrar uma sequência de árvores na qual cada componente apresente uma evolução em comparação com a anterior. Esse processo representa o aprendizado contínuo entre as árvores. Portanto, de acordo com Kern, Klausch e Kreuter (2019), cada árvore tem por objetivo principal melhorar os valores preditos pela anterior, corrigindo os seus erros. Portanto, assim como o Random Forest, o Gradient Boosting e suas evoluções produzem predições em conjunto, que usualmente superam a predição de uma única árvore de decisão.

Segundo Santhanam et al. (2017), o conceito de gradient boosting envolve três etapas básicas. Primeiramente, deve ser identificada uma função de perda (loss function) diferenciável compatível com o problema a ser solucionado. Uma vantagem de modelos baseados em gradient boosting é que os mesmos são adaptáveis a diferentes funções de perda, isto é, não é necessário que um novo algoritmo seja derivado, desde que a função de perda escolhida seja adequada e, portanto, seja incorporada à estrutura final.

Cria-se um aprendiz fraco (weak learner) para fazer predições. No contexto de gradient boosting, há árvores de regressão, que produzem saídas de valor real para as divisões cujas saídas podem ser somadas, permitindo que saídas subsequentes de diferentes modelos sejam adicionadas. Essa abordagem permite a melhoria contínua dos resíduos nas predições, levando a resultados mais precisos. Sendo assim, estas árvores são criadas de maneira gulosa<sup>3</sup>, o que urge a frequente necessidade de impor restrições para que os aprendizes fracos sigam sendo fracos e que as árvores continuem sendo criadas de maneira gulosa (SANTHANAM et al., 2017).

Por fim, cria-se um modelo aditivo, que soma as predições dos aprendizes fracos, em prol da redução da função de perda. As árvores são adicionadas uma por vez. Então, a saída

---

<sup>3</sup> Estratégia de resolução de problemas onde se faz o que se considera a melhor escolha para o momento, sem considerar as implicações futuras

produzida na nova árvore é adicionada à saída da sequência de árvores pré-existentes, visando a evolução da saída final do modelo. Esse processo é finalizado quando o valor otimizado apropriado para a função de perda é encontrado (SANTHANAM et al., 2017).

### **XGBoost**

Em prol da otimização do algoritmo de gradient boosting simples, Chen e Guestrin (2016) criaram o XGBoost, uma implementação que possui algumas nuances que o diferem do seu antecessor. Por mais que o XGBoost seja, por essência, um modelo baseado em gradient boosting, o mesmo se destaca por realizar o processo de adição de aprendizes fracos de maneira não sequencial, isto é, um após o outro, mas sim por uma abordagem multi-threaded, onde utiliza-se adequadamente os núcleos da CPU da máquina, levando a ganhos de velocidade e desempenho. Ademais, o XGBoost também apresenta a implementação ciente de esparsidade (sparse aware implementation), onde são incluídos: o tratamento automático de valores faltantes, estrutura de blocos, capaz de suportar a paralelização da construção de árvores, e o processo de treinamento continuado, que impulsiona um modelo já ajustado para novos dados não vistos (SANTHANAM et al., 2017).

### **LightGBM**

Outra otimização do gradient boosting é o algoritmo conhecido como LightGBM, introduzido por Ke et al. (2017). O mesmo foi desenvolvido com foco na eficiência computacional, ao mesmo tempo que apresenta um nível satisfatório de precisão (até 20 vezes mais rápido que o gradient boosting simples). O termo Light vem justamente da capacidade desse algoritmo de realizar suas tarefas, tanto de treino, teste e validação, de maneira mais veloz quando comparado a outros algoritmos de boosting (THAI, 2022).

A principal peculiaridade do LightGBM é a maneira como a árvore é expandida. O LightGBM utiliza a estratégia conhecida como leaf-wise (ou “melhor primeiro”), onde a folha com maior perda é a escolhida como o destino de crescimento. Assim, a árvore cresce na mesma folha, o que pode reduzir mais a perda, resultando em uma maior precisão, mesmo que gere uma árvore desbalanceada. Essa estratégia pode se mostrar mais precisa do que a level-

wise (depth-first ou “profundidade primeiro”), utilizada tradicionalmente por outros modelos de boosting (THAI, 2022).

Ademais, é importante ressaltar outras características que potencializam a velocidade do LightGBM: Gradient-based One-Side Sampling (GOSS) (Amostragem Unilateral Baseada em Gradiente, em português) e Exclusive Feature Bundling (EFB) (Pacote de recursos exclusivos, em português). O GOSS é uma técnica de subsampling (subamostragem) que pode reduzir drasticamente o número de instâncias, ao realizar amostragem aleatória somente em instâncias dos pequenos gradientes, enquanto o EFB é uma técnica de fusão, que visa reduzir o número de atributos. Ambas as técnicas são importantes para acelerar o processo de treinamento do modelo, uma vez que o tempo computacional para encontrar o ponto de divisão é proporcional ao número de instâncias e features (THAI, 2022).

Cabe ressaltar que como o LightGBM foi idealizado para trabalhar com grande volume de dados, o mesmo apresenta alta sensibilidade a dados pequenos, o que aumenta o risco do modelo sofrer overfitting (sobreajuste). Desta forma, a escolha pelos três algoritmos se deve ao forte potencial preditivo e às semelhanças entre os modelos, que apresentam uma alta capacidade de resolução para a problemática abordada, além de facilitar uma análise comparativa do desempenho entre os três. A seguir, serão apresentadas as métricas de avaliação a serem utilizadas para medir e comparar o desempenho de cada modelo (THAI, 2022).

## **2.4 Métricas de Avaliação**

Para todos os modelos de aprendizado de máquina, a avaliação do desempenho é um passo fundamental para qualificar a precisão do algoritmo para realizar a tarefa desejada. As métricas de avaliação são fundamentais para medir a capacidade de generalização desses modelos sob dados não vistos. Com o objetivo de prever um valor contínuo. O desempenho de modelos em tarefas de regressão pode ser avaliado por métricas que medem a diferença (erro) entre os valores preditos e os valores reais (observados). As métricas utilizadas para avaliação dos modelos selecionados para o desenvolvimento do projeto serão explicadas a seguir. (DEMIRTRK; MINTEMUR; ARSLAN, 2025)



### Erro Absoluto Médio (MAE)

A métrica Mean Absolute Error (MAE) ou Erro Absoluto Médio é utilizada para estimar a diferença absoluta média entre os resultados das previsões do modelo para a tarefa desejada. Portanto, através dessa métrica, assim como as outras a serem apresentadas em sequência, é possível medir o quanto o modelo errou em relação ao real valor do atributo de saída. Essa métrica torna-se de alta utilidade para comparar o desempenho de diferentes modelos em uma tarefa de regressão. A fórmula da MAE é apresentada na Equação 2.4 (BELYADI; HAGHIGHAT, 2021).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.4)$$

### Erro Quadrático Médio (MSE)

Outra métrica amplamente utilizada na avaliação de modelos é a Mean Squared Error (MSE) ou Erro Quadrático Médio. A mesma representa a função de perda de  $L_2$ , que retrata o cálculo da média das diferenças quadráticas entre as previsões e os valores reais. Desta forma, assim como a MAE, essa métrica pode ser utilizada para compreender a diferença entre o que foi predito pelo modelo e o que representa os valores reais (BELYADI; HAGHIGHAT, 2021).

Portanto, o objetivo dos modelos durante o treinamento, para ambas as métricas, é minimizar os valores das mesmas, buscando valores o mais próximos de zero, que indicariam melhor ajuste dos modelos aos dados. Vale lembrar que o MSE é mais sensível a outliers quando comparado ao MAE, o que lhe dá certa vantagem em aplicações no mundo real (BELYADI; HAGHIGHAT, 2021; CHICCO; WARRENS; JURMAN, 2021). A Equação 2.5 formaliza como o MSE é calculado.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.5)$$

### Raiz do Erro Quadrático Médio (RMSE)

Além do MSE, a Root Mean Squared Error (RMSE) ou Raiz do Erro Quadrático Médio, que simplesmente representa a raiz quadrada do MSE, foi utilizada para a avaliação dos modelos. De acordo com Plevris et al. (2022), diferente do MSE, essa mantém a unidade de

medida utilizada na mesma escala do atributo de saída. Assim como o MAE e o MSE, valores menores de RMSE, ou seja, próximos de 0, representam um erro menor na predição realizada pelos modelos avaliados. O cálculo dessa métrica ocorre conforme a Equação 2.6.

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.6)$$

### **Coefficiente de Determinação ( $R^2$ )**

Por fim, foi utilizado o Coefficient of Determination ( $R^2$ ) ou Coeficiente de Determinação, uma medida de ajuste responsável por indicar o quão proporcional é a variância na predição do atributo de saída a partir dos atributos de entrada. Assim como as métricas abordadas anteriormente, esta também quantifica o quão satisfatório foi o ajuste de modelos de regressão aos dados observados. Entretanto, segundo Chicco, Warrens e Jurman (2021), para o  $R^2$ , diferente das métricas abordadas anteriormente, quanto maior o valor, melhor foi a performance. Seu pior resultado possível seria representado por  $-\infty$ , enquanto seu melhor resultado seria 1. A fórmula do  $R^2$  está exposta na Equação 2.7.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (2.7)$$

## **2.5 Testes Estatísticos**

A aplicação e a comparação das métricas de avaliação dos modelos são fundamentais para compreender as diferenças de desempenho entre eles em uma mesma tarefa. Entretanto, é importante entender que, embora seja quase inevitável haver uma ligeira diferença entre os resultados obtidos pela avaliação entre diferentes modelos, é crucial determinar se essa disparidade é significativa. É importante entender se tal diferença é grande o suficiente para ser considerada uma evidência real de que os valores são dissimilares, ou se decorre apenas de variação aleatória (RAINIO; TEUHO; KLÉN, 2024).

O processo de realização de um teste estatístico pode ser resumido em algumas etapas: primeiramente, formula-se a hipótese nula ( $H_0$ ), que pressupõe a inexistência de uma diferença real. Em seguida, define-se um nível de significância ( $\alpha$ )  $\in (0, 1)$  e estabelece-se uma estatís-

tica de teste ( $Z$ ) adequada, com sua respectiva distribuição de probabilidade  $P(Z | H_0)$  sob a hipótese nula. Por fim, essa distribuição é utilizada para calcular a probabilidade de se obter um valor para  $Z$  que seja, no mínimo, tão extremo quanto o valor  $z$  já observado. Assim, caso a probabilidade resultante, com a equação representada em Equação 2.8, seja menor que  $\alpha$ , a hipótese nula é rejeitada, isto é, a diferença é estatisticamente significativa (RAINIO; TEUHO; KLÉN, 2024).

$$p = 2 \min \{P(Z \leq z | H_0), P(Z \geq z | H_0)\} \quad (2.8)$$

É necessário definir o tipo de erro cometido. Ao rejeitar uma hipótese nula verdadeira, foi cometido um erro do Tipo 1. O cenário inverso caracteriza um erro do Tipo 2. É possível controlar a probabilidade de um erro do Tipo 1, uma vez que ela seria igual a  $\alpha$ . Para além do uso de  $p$ , é possível usar  $\alpha$  para calcular os valores críticos para a estatística, o que possibilita decidir se a hipótese nula será aceita ou rejeitada. Para o presente trabalho, os testes estatísticos selecionados foram: Friedman e Nemenyi (RAINIO; TEUHO; KLÉN, 2024).

Para situações onde se deve comparar três ou mais amostras relacionadas<sup>4</sup>, a Estatística oferece procedimentos especializados para testar a significância das diferenças entre elas. Os testes utilizados para o presente projeto serão apresentados a seguir (RAINIO; TEUHO; KLÉN, 2024).

### Teste de Friedman

Originalmente proposto por Friedman (1937), o Teste de Friedman é um teste não-paramétrico, o qual ranqueia os algoritmos para cada conjunto de dados separadamente, onde o melhor modelo, em termos de desempenho, recebe o rank 1, o segundo melhor recebe o rank 2, e assim por diante. Em caso de empate, são atribuídos ranks médios. Seja  $r_j^i$  o rank do  $j$ -ésimo dos  $k$  algoritmos no  $i$ -ésimo dos  $N$  conjuntos de dados. O teste de Friedman compara os ranks médios dos algoritmos,  $R_j$ , conforme demonstrado em Equação 2.9.

<sup>4</sup> Amostras relacionadas referem-se aos desempenhos dos modelos de regressão medidos nos mesmos conjuntos de dados, preferencialmente utilizando as mesmas divisões entre conjuntos de treino, teste e avaliação

$$R_j = \frac{1}{N} \sum_i r_j^i \quad (2.9)$$

De acordo com a hipótese nula, onde todos os algoritmos são equivalentes e, portanto, possuem ranks  $R_j$  iguais, a estatística de Friedman é dada conforme a Equação 2.10.

$$\chi_F^2 = \frac{12N}{k(k+1)} \left[ \sum_j R_j^2 - \frac{k(k+1)^2}{4} \right] \quad (2.10)$$

A estatística  $\chi_F^2$  é distribuída de acordo com a distribuição  $\chi^2$  com  $k - 1$  graus de liberdade, quando  $N$  e  $k$  são grandes o suficiente (como regra prática,  $N > 10$  e  $k > 5$ ). Para um número menor de algoritmos e conjuntos de dados, valores críticos exatos devem ser calculados. Portanto, caso a hipótese nula seja rejeitada no teste de Friedman, deve-se prosseguir com um teste *post-hoc* conhecido como Nemenyi, a ser apresentado a seguir (RAINIO; TEUHO; KLÉN, 2024).

### Teste *Post-Hoc* de Nemenyi

O Teste *Post-Hoc* de Nemenyi, inicialmente proposto por Nemenyi (1963), é utilizado para comparar todos os modelos entre si. Neste caso, a diferença de desempenho entre dois algoritmos é considerada significativa quando os seus ranks médios correspondentes diferirem em pelo menos a Diferença Crítica (CD), calculada conforme mostra a Equação 2.11 (RAINIO; TEUHO; KLÉN, 2024).

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}} \quad (2.11)$$

De forma que  $k$  é o número de algoritmos que estão a ser comparados;  $N$  é o número de conjuntos de dados;  $q_\alpha$  é o valor crítico baseado na estatística do *Studentized range* (ou intervalo estudentizado, em português), ajustado para o nível de significância  $\alpha$ .

### Análise de Resíduos

Os resíduos (ou erros) desempenham um papel fundamental na avaliação de algoritmos de aprendizado de máquina. Em qualquer estrutura preditiva, os resíduos são a diferença entre o

valor observado e a predição do modelo. Desta forma, através da análise dos mesmos, é possível diagnosticar problemas e melhorar o desempenho dos algoritmos (VERMA, 2025).

É possível gerar gráficos que mostrem a relação entre os resíduos e as predições (e também entre os resíduos e os valores observados), de forma que um modelo bem treinado deve mostrar resíduos dispersos aleatoriamente e sem padrões discerníveis. Através da forma como os resíduos estão distribuídos, é possível extrair alguns insights importantes. Caso um algoritmo apresente uma estrutura sistemática, enquanto outro apresente resíduos distribuídos de forma mais uniforme, o segundo é preferido. Desta forma, é possível utilizar esses gráficos para complementar a comparação entre diferentes algoritmos, no que diz respeito ao quão bem treinados eles estão para prever o valor desejado (VERMA, 2025).

É válido ressaltar a possível ocorrência da heterocedasticidade para modelos de regressão. Esse evento ocorre quando a variância dos resíduos não é constante em todos os níveis de um atributo de saída em um algoritmo de regressão. Isso pode resultar em erros padrão não confiáveis, testes de hipótese enganosos e estimativas ineficientes (VERMA, 2025).

Assim, é possível detectar a heterocedasticidade através dos gráficos de resíduos vs. predições. Caso os resíduos apresentem um padrão sistemático, como um formato de leque onde a variância aumenta com as predições, a heterocedasticidade está presente. É possível tratar a heterocedasticidade de diferentes maneiras, através de técnicas como a transformação logarítmica ou de raiz quadrada, como forma de estabilizar a variância. Assim, identificar e corrigir a heterocedasticidade é fundamental para tornar os algoritmos mais confiáveis e generalizáveis para aplicações no mundo real (VERMA, 2025).

### 3 METODOLOGIA

Este capítulo detalha a metodologia empregada no projeto, desde a aquisição e tratamento dos dados até a avaliação e comparação dos modelos, seguida pelos testes de significância estatística. Os detalhes de cada fase do processo são apresentados nas seções subsequentes.

#### 3.1 Conjunto de Dados

O conjunto de dados utilizado nesse estudo provém de uma comunidade online conhecida como Kaggle, amplamente utilizada pela comunidade de ciência de dados para encontrar datasets, publicar e trabalhar em conjunto com outros cientistas de dados e engenheiros de aprendizado de máquina (KAGGLE, 2025). O mesmo reúne um grande volume de informações sobre acomodações do Airbnb localizadas em Sydney e região, na Austrália, incluindo características como preço da diária, tipo, número de quartos, comodidades, nome, descrição, entre outras. A abrangência e riqueza de dados desse conjunto o tornam um ponto de partida interessante para a realização das etapas da metodologia detalhada nesse capítulo.

#### 3.2 Ambiente de Implementação

A modelagem e avaliação dos algoritmos descritos nesse projeto foram realizados na linguagem de programação Python (versão 3.12.12), através do Google Colab, serviço hospedado do Jupyter Notebook, tecnologia open source que permite a combinação entre código, texto e visualizações em um mesmo documento e que não requer quaisquer configurações para ser utilizado e fornece acesso gratuito a recursos computacionais, como GPUs e TPUs. A escolha dessa combinação se deu pela sua natureza baseada em nuvem, que mitiga a necessidade de alto poder de processamento em hardware local (GOOGLE, 2025).

#### 3.3 Pipeline

Desenvolver modelos de aprendizado de máquina que sejam robustos e escaláveis pode ser um desafio (BDAIR, 2022), assim, os pipelines emergiram como uma solução para esses problemas. Um pipeline de aprendizado de máquina encapsula todo o fluxo de trabalho de um

projeto, desde o pré-processamento de dados até o treinamento e avaliação dos modelos, ao fornecer uma estrutura para organizar todo o processo. Portanto, esse conceito consiste em uma série de componentes vitais ao pleno desenvolvimento de um projeto robusto. Os elementos fundamentais ao desenvolvimento dessa pesquisa serão apresentados a seguir.

### **Coleta de Dados**

A coleta de dados envolve reunir dados não processados obtidos por diferentes fontes, como conjuntos de dados, documentos de texto, imagens ou vídeos. É fundamental que os dados sejam relevantes à problemática a ser solucionada e os mesmos podem necessitar de pré-processamento antes de alimentarem o pipeline (BDAIR, 2022).

### **Pré-processamento de Dados**

O pré-processamento de dados inclui a limpeza, transformação e preparação de dados não processados para que possam ser utilizados como base do treinamento de modelos de aprendizado de máquina. Assim, essa etapa inclui processos como tratar valores faltantes e anomalias, escalar atributos de entrada, codificar atributos categóricos, entre outras transformações que tornam os dados adequados ao treinamento (BDAIR, 2022).

### **Engenharia de Atributos**

A engenharia de atributos foca na criação de novos atributos de entrada ou transformação daqueles já existentes no conjunto de dados original, em busca de aprimorar o poder preditivo dos modelos. Essa etapa exige um processo criativo e expertise para extrair informações dos dados. As técnicas de engenharia de atributos podem incluir redução de dimensionalidade, seleção de atributos de entrada e criação de novos atributos de interação (BDAIR, 2022).

### **Ajuste de Hiperparâmetros**

Muitos modelos de aprendizado de máquina possuem hiperparâmetros que podem ser otimizados em prol do desempenho dos algoritmos. Esse processo envolve a busca sistemática,

em um determinado espaço de hiperparâmetros, para encontrar a melhor combinação possível, isto é, o conjunto que produza os melhores resultados, avaliados através de alguma métrica de avaliação. Para esse processo, podem ser utilizadas técnicas como a busca em grade (Grid search), busca aleatória (Random search) ou otimização bayesiana (Bayesian optimization) (BDAIR, 2022).

### **Treinamento do Modelo**

Um algoritmo de aprendizado de máquina é treinado sobre um conjunto de dados numéricos pré-processados, para aprender padrões subjacentes e relacionamentos entre os atributos de entrada. A escolha do algoritmo depende do tipo de problema abordado (classificação, regressão, agrupamento, etc.) e das características dos dados. Assim, o modelo é treinado utilizando diversas técnicas de otimização, em busca da minimização de erros e potencialização das métricas de desempenho do algoritmo (BDAIR, 2022).

### **Avaliação do Modelo**

Após o treinamento, o modelo deve ser avaliado para assegurar seu desempenho e capacidade de generalização. Esse processo envolve utilizar métricas de avaliação, como as apresentadas no capítulo anterior. Tal procedimento é fundamental para identificar pontos fortes e fracos do modelo, guiando possíveis melhorias no fluxo de trabalho (BDAIR, 2022).



## 4 RESULTADOS E DISCUSSÃO

Neste capítulo, são apresentados os resultados do projeto proposto, focado na comparação da capacidade preditiva de três modelos que são o LightGBM, Random Forest e XGBoost, para estimar o valor de acomodações do Airbnb, juntamente com a análise geoespacial. O código desenvolvido na realização do projeto está publicamente disponível<sup>1</sup>.

A priori, logo após a obtenção da base de dados, encontrada em uma publicação de um usuário do Kaggle<sup>2</sup>, foi realizado o processo de leitura do arquivo (disponibilizado em formato .csv) e o entendimento dos 83 atributos de entrada. O dicionário de dados é detalhado na Tabela 4.1.

Tabela 4.1 – Dicionário de dados das acomodações do Airbnb

| Nome do Atributo    | Significado do Atributo                        | Tipo de Dado |
|---------------------|------------------------------------------------|--------------|
| name                | Título do anúncio da acomodação.               | object       |
| summary             | Breve resumo do anúncio.                       | object       |
| space               | Descrição mais detalhada do espaço disponível. | object       |
| description         | Descrição completa do anúncio.                 | object       |
| experiences_offered | Tipo de experiências oferecidas.               | object       |
| neigh_overview      | Descrição da vizinhança pelo anfitrião.        | object       |
| notes               | Notas adicionais do anfitrião.                 | object       |
| transit             | Informações de transporte público.             | object       |
| access              | Informações sobre acesso à acomodação.         | object       |
| interaction         | Nível de interação com o anfitrião.            | object       |
| house_rules         | Regras da casa.                                | object       |
| host_id             | Identificador único do anfitrião.              | int64        |
| host_since          | Data de registro do anfitrião.                 | object       |
| host_location       | Localização geográfica do anfitrião.           | object       |
| host_about          | Texto de apresentação do anfitrião.            | object       |

<sup>1</sup> <<https://github.com/joaoadn/airbnb-price-prediction>>

<sup>2</sup> Base de dados disponível em: <<https://www.kaggle.com/datasets/berrywhite/airbnb/data>>, acessado em 24/09/2025

| Nome do Atributo       | Significado do Atributo                         | Tipo de Dado |
|------------------------|-------------------------------------------------|--------------|
| host_response_time     | Tempo médio de resposta do anfitrião.           | object       |
| host_response_rate     | Taxa de resposta do anfitrião (em %).           | object       |
| host_acceptance_rate   | Taxa de aceitação de pedidos de reserva (em %). | float64      |
| host_is_superhost      | Indica se é Superhost (t/f).                    | object       |
| host_neighbourhood     | Vizinhança do anfitrião.                        | object       |
| host_listings_count    | No. de anúncios ativos do anfitrião na cidade.  | float64      |
| host_total_listings    | No. total de anúncios ativos na plataforma.     | float64      |
| host_verifications     | Lista de verificações de identidade.            | object       |
| host_identity_verified | Identidade do anfitrião verificada (t/f).       | object       |
| street                 | Nome da rua da acomodação.                      | object       |
| neighbourhood          | Nome da vizinhança (anfitrião).                 | object       |
| neigh_cleansed         | Nome padronizado da vizinhança.                 | object       |
| city                   | Cidade da acomodação.                           | object       |
| zipcode                | Código postal.                                  | object       |
| smart_location         | Localização inteligente.                        | object       |
| latitude               | Coordenada geográfica de latitude.              | float64      |
| longitude              | Coordenada geográfica de longitude.             | float64      |
| is_location_exact      | Indica se a localização é exata (t/f).          | object       |
| property_type          | Tipo de propriedade.                            | object       |
| room_type              | Tipo de quarto/espço alugado.                   | object       |
| accommodates           | No. máximo de hóspedes.                         | int64        |
| bathrooms              | Número de banheiros.                            | float64      |
| bedrooms               | Número de quartos.                              | float64      |
| beds                   | Número de camas.                                | float64      |
| bed_type               | Tipo de cama.                                   | object       |
| amenities              | Lista de comodidades disponíveis.               | object       |
| square_feet            | Tamanho da propriedade em pés quadrados.        | float64      |
| weekly_discount        | Desconto para estadias semanais.                | float64      |

| <b>Nome do Atributo</b> | <b>Significado do Atributo</b>                     | <b>Tipo de Dado</b> |
|-------------------------|----------------------------------------------------|---------------------|
| monthly_discount        | Desconto para estadias mensais.                    | float64             |
| security_deposit_perc   | Percentual do depósito de segurança.               | float64             |
| cleaning_fee_perc       | Percentual da taxa de limpeza.                     | float64             |
| guests_included         | No. de hóspedes incluídos no preço base.           | int64               |
| extra_people_perc       | Percentual cobrado por hóspede extra.              | float64             |
| minimum_nights          | No. mínimo de noites para reserva.                 | int64               |
| maximum_nights          | No. máximo de noites para reserva.                 | int64               |
| min_min_nights          | Menor valor de noites mínimas (ao longo do tempo). | int64               |
| max_min_nights          | Maior valor de noites mínimas (ao longo do tempo). | int64               |
| min_max_nights          | Menor valor de noites máximas (ao longo do tempo). | int64               |
| max_max_nights          | Maior valor de noites máximas (ao longo do tempo). | int64               |
| min_nights_avg_ntm      | Média de noites mínimas (12 meses).                | float64             |
| max_nights_avg_ntm      | Média de noites máximas (12 meses).                | float64             |
| availability_30         | Dias disponíveis nos próximos 30 dias.             | int64               |
| availability_60         | Dias disponíveis nos próximos 60 dias.             | int64               |
| availability_90         | Dias disponíveis nos próximos 90 dias.             | int64               |
| availability_365        | Dias disponíveis nos próximos 365 dias.            | int64               |
| number_of_reviews       | Número total de avaliações.                        | int64               |
| reviews_ltm             | No. de avaliações nos últimos 12 meses.            | int64               |
| first_review            | Data da primeira avaliação.                        | object              |
| last_review             | Data da última avaliação.                          | object              |
| review_scores_rating    | Nota geral de avaliação (0 a 100).                 | float64             |
| review_scores_acc       | Nota de precisão/fidelidade (0 a 10).              | float64             |
| review_scores_clean     | Nota de limpeza (0 a 10).                          | float64             |

| Nome do Atributo    | Significado do Atributo                          | Tipo de Dado |
|---------------------|--------------------------------------------------|--------------|
| review_scores_check | Nota de facilidade de check-in (0 a 10).         | float64      |
| review_scores_comm  | Nota de comunicação (0 a 10).                    | float64      |
| review_scores_loc   | Nota da localização (0 a 10).                    | float64      |
| review_scores_value | Nota de custo-benefício (0 a 10).                | float64      |
| requires_license    | Se a acomodação requer licença (t/f).            | object       |
| instant_bookable    | Se a reserva é instantânea (t/f).                | object       |
| is_business_travel  | Pronta para viagens de negócios (t/f).           | object       |
| cancellation_policy | Política de cancelamento aplicada.               | object       |
| require_guest_pic   | Requer foto de perfil do hóspede (t/f).          | object       |
| require_guest_phone | Requer verificação de telefone (t/f).            | object       |
| calc_host_listings  | Contagem de anúncios do anfitrião (calculada).   | int64        |
| calc_entire_homes   | Contagem de casas/apts inteiros do anfitrião.    | int64        |
| calc_private_rooms  | Contagem de quartos privados do anfitrião.       | int64        |
| calc_shared_rooms   | Contagem de quartos compartilhados do anfitrião. | int64        |
| reviews_per_month   | Média de avaliações por mês.                     | float64      |
| price               | Preço da acomodação (Atributo de saída).         | object       |

#### 4.1 Pré-processamento dos dados

A fase inicial do pré-processamento focou na limpeza e padronização dos atributos. O processo iniciou-se com a conversão do atributo de saída “price” em valor numérico de ponto flutuante, através da remoção de caracteres não numéricos, como “\$” e “,”.

Foi realizada a conversão de atributos de tempo, onde: *host\_since*, *first\_review* e *last\_review* foram iterados e convertidos para o formato de data e hora da biblioteca Pandas. Essas etapas garantiram que atributos importantes fossem devidamente tratados antes de se executar as etapas subsequentes.

A etapa seguinte consistiu na eliminação de 17 atributos de entrada da base de dados. Esta remoção foi motivada pela avaliação de que estes atributos não seriam úteis ou seriam re-

dundantes para o objetivo da modelagem preditiva. O descarte ocorreu devido a três fatores: atributos textuais (como “summary” e “description”) foram removidas pela alta complexidade computacional de sua codificação e por representarem informações descritivas que já eram capturadas por outros atributos; outros atributos foram excluídos por apresentarem baixa ou zero variância de valores; e, por fim, o atributo “host\_acceptance\_rate” foi removida por conter apenas dados nulos. Esta etapa de limpeza e redução resultou em um conjunto de dados final contendo 66 atributos de entrada.

Após a execução das etapas anteriores, foram identificadas no conjunto de dados 9.838 instâncias e 67 atributos de entrada. Dentre elas, foram encontradas: 20 atributos categóricos (object), 19 atributos numéricos inteiros, 25 de ponto flutuante e 3 atributos de tempo. Para complementar a análise, foram identificados os atributos com  $> 1\%$  de dados faltantes e foi realizada uma análise estatística do atributo de saída. Os resultados são apresentados na Tabela 4.2 e Tabela 4.3.

Tabela 4.2 – Atributos com dados faltantes (Percentual  $> 1\%$ )

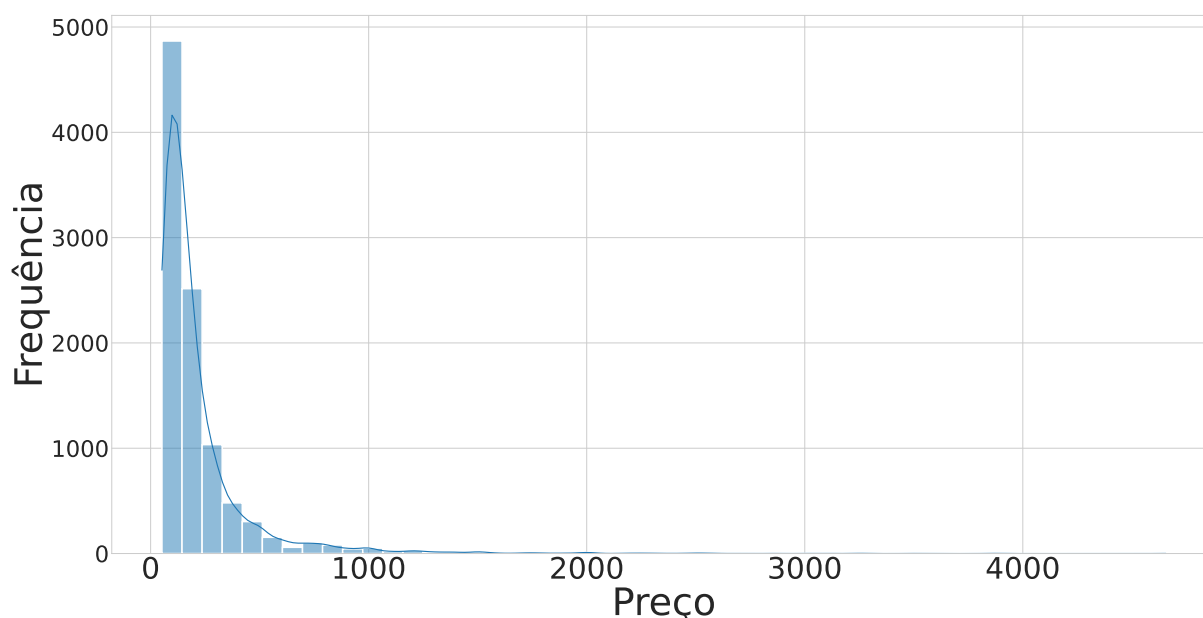
| Atributo                  | Contagem | Percentual(%) |
|---------------------------|----------|---------------|
| square_feet               | 9779     | 99.40         |
| monthly_discount          | 9299     | 94.52         |
| weekly_discount           | 8940     | 90.87         |
| host_resp_time            | 4566     | 46.41         |
| host_resp_rate            | 4566     | 46.41         |
| host_about                | 4363     | 44.35         |
| sec_deposit_perc          | 3303     | 33.57         |
| review_scores_value       | 2962     | 30.11         |
| review_scores_location    | 2960     | 30.09         |
| review_scores_checkin     | 2959     | 30.08         |
| review_scores_accuracy    | 2957     | 30.06         |
| review_scores_comm        | 2956     | 30.05         |
| review_scores_cleanliness | 2954     | 30.03         |
| review_scores_rating      | 2950     | 29.99         |
| host_neigh                | 2915     | 29.63         |
| first_review              | 2698     | 27.42         |
| last_review               | 2698     | 27.42         |
| reviews_per_month         | 2698     | 27.42         |
| cleaning_fee_perc         | 2445     | 24.85         |

Tabela 4.3 – Resumo Estatístico do atributo de saída

| Atributo | Média  | Desvio-Padrão | Valor Mínimo | Valor Máximo |
|----------|--------|---------------|--------------|--------------|
| price    | 214.68 | 250.72        | 52.00        | 4656.00      |

No que tange à detecção de anomalias, foi gerado um histograma, representado na Figura 4.1, que mostra que os dados apresentam uma curva dissemelhante ao formato de sino, o que imediatamente indica a anormalidade da distribuição do atributo de saída. Além disso, o diagrama de caixa Figura 4.2 revela que existem observações além dos “bigodes” do gráfico, o que as caracteriza como anomalias. Além disso, é possível perceber a dispersão entre os dados, onde os valores mais altos se encontraram mais isolados à direita (DAS, 2016).

Figura 4.1 – Histograma do atributo de saída



O gráfico em Figura 4.3 mostra que os pontos se desviam da linha e constroem uma curva da esquerda pra direita, o que aponta para uma distribuição não normal (DAS, 2016).

Para complementar a análise de normalidade, foi realizado o teste de Shapiro-Wilk para o atributo de saída. Os resultados do teste confirmaram a distribuição não-normal dos dados, uma vez que o  $p$ -valor encontrado foi de 0.00, o que rejeita a hipótese nula por  $0.00 < 0.05$ .

Posteriormente, foi gerado um mapa de calor de correlação entre os atributos numéricos, encontrado em Figura 4.4. Através dessa análise, foi possível identificar quais atributos possuem alta correlação com o preço, o que os classifica como fortes candidatos a serem altamente preditivos para o atributo de saída.

Em sequência, foi realizada uma análise dos atributos com variância zero (isto é, atributos constantes) e daquelas com baixa variância (possuindo apenas 2 ou 3 valores únicos). Esta

Figura 4.2 – Diagrama de caixa do atributo de saída

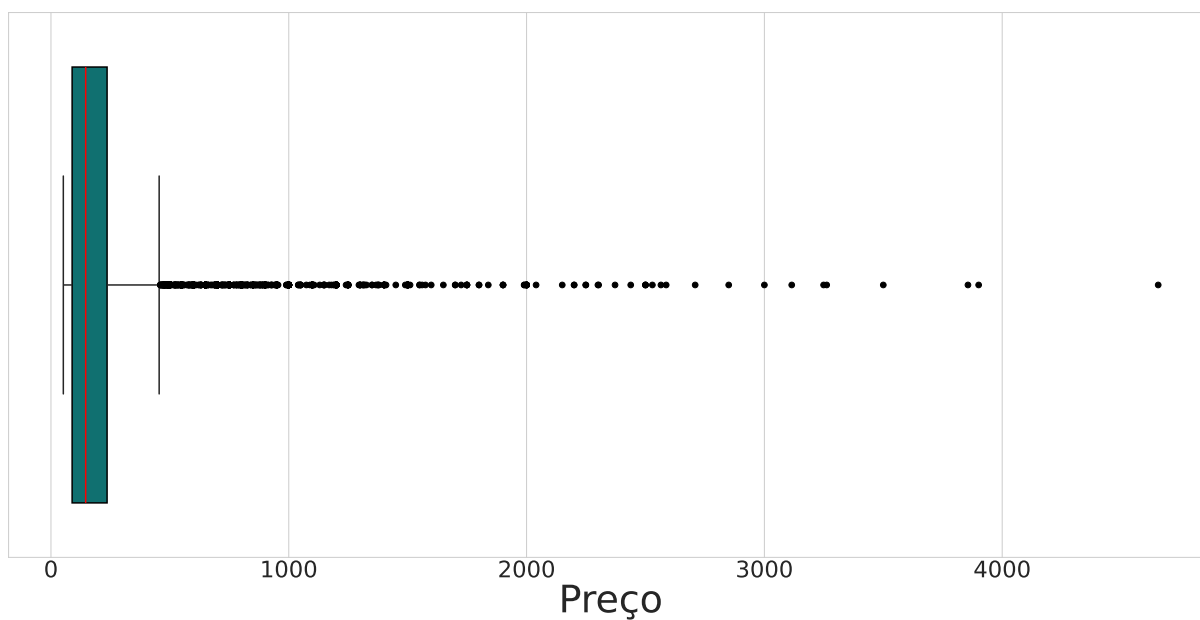
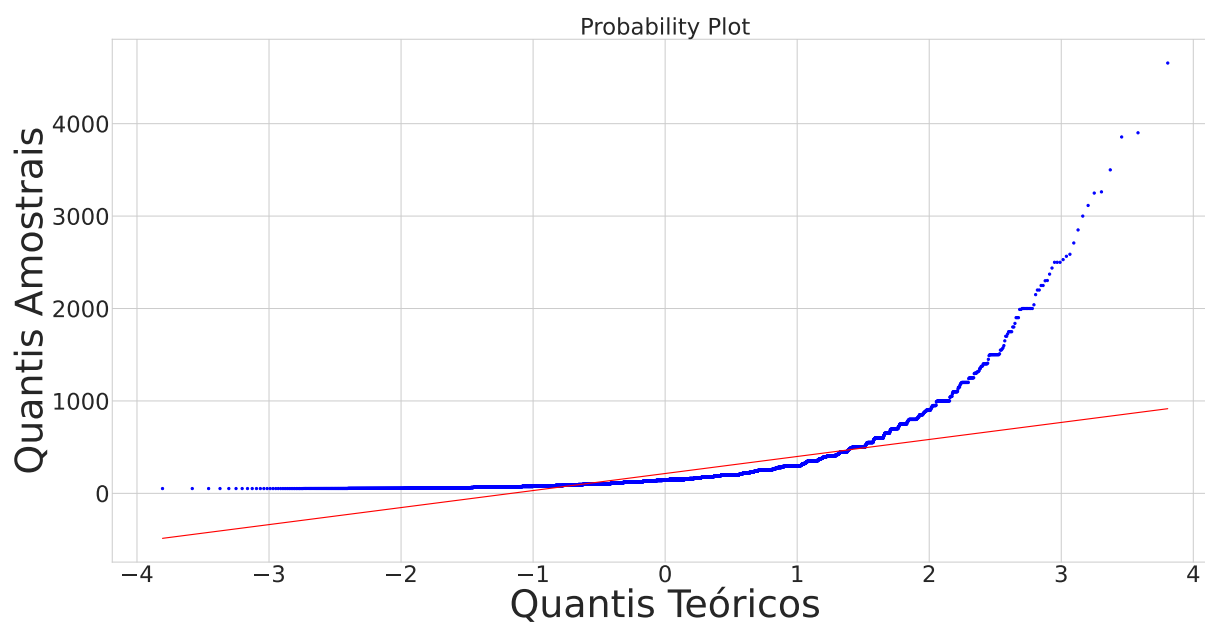


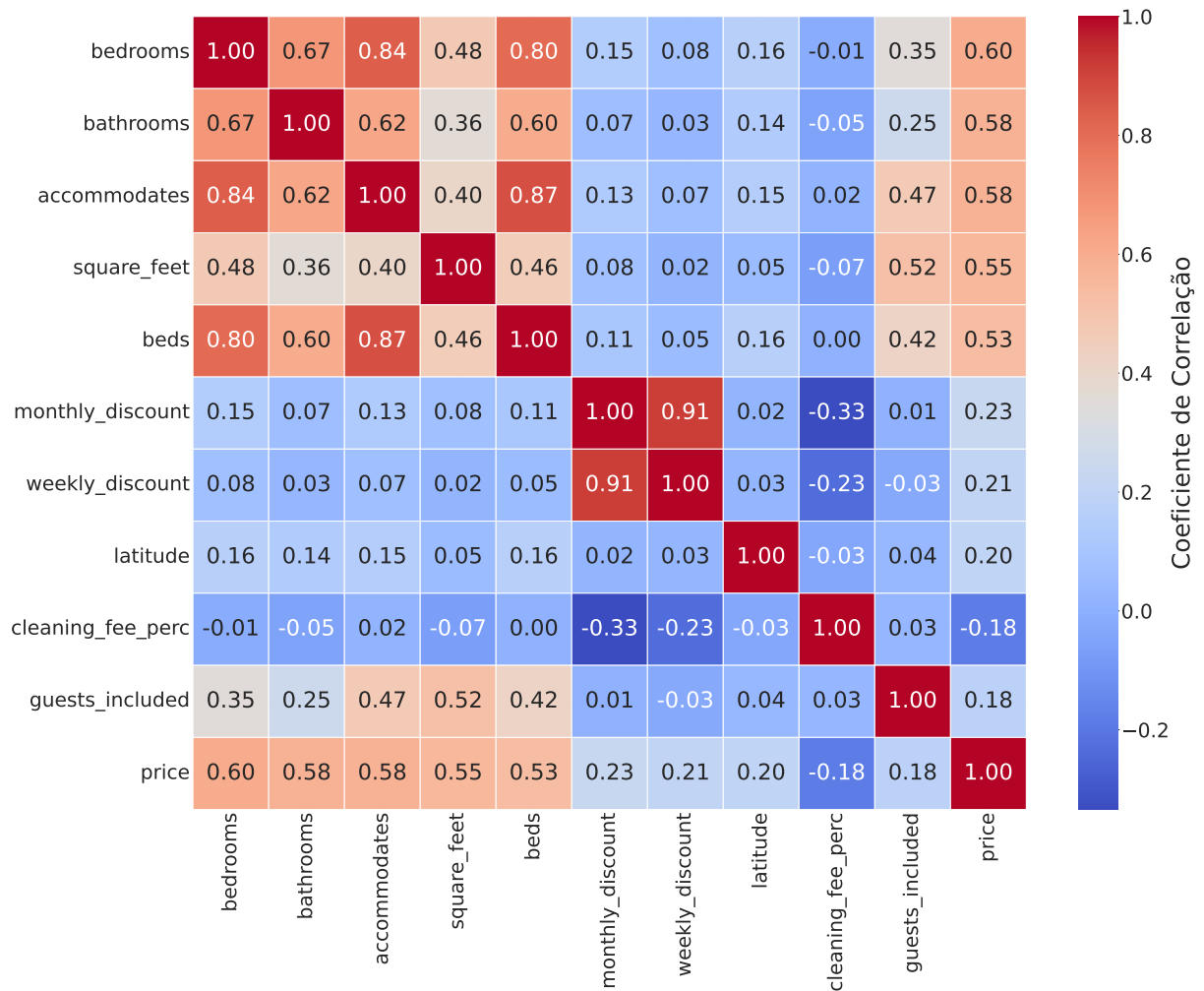
Figura 4.3 – Q-Q Plot do atributo de saída



etapa foi crucial para compreender a importância preditiva desses atributos e serviu como base metodológica para a engenharia de features realizada posteriormente.

Como resultado, identificou-se que *is\_business\_travel\_ready* é um atributo constante, com todas as acomodações possuindo o valor false, o que descarta seu poder preditivo. Além disso, notou-se que os atributos *host\_is\_superhost*, *host\_identity\_verified*, *is\_location\_exact*,

Figura 4.4 – Mapa de calor de Correlação de Pearson

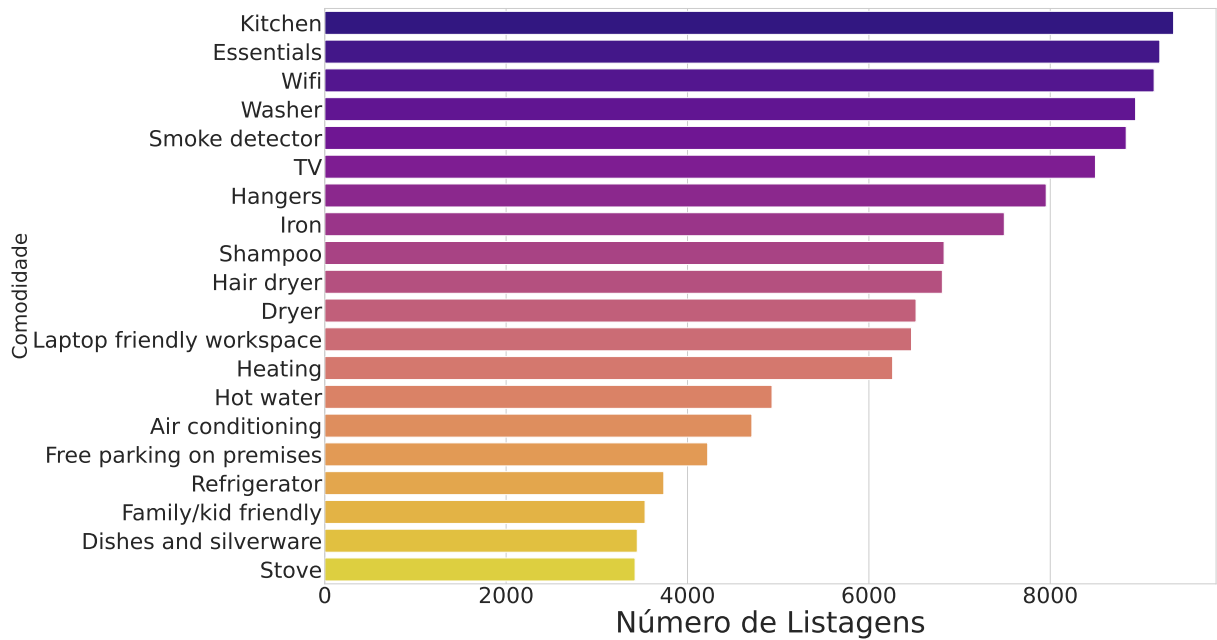


*room\_type*, *instant\_bookable*, *require\_guest\_profile\_picture* e *require\_guest\_phone\_verification* possuem baixa variância. Contudo, essa característica não se mostrou um problema, pois era esperado que tais atributos possuísem um número reduzido de valores únicos devido à sua natureza predominantemente binária.

Na etapa seguinte, e também com o objetivo de fornecer uma base metodológica para a engenharia de atributos, foi feito o ranqueamento das 20 comodidades mais comuns entre as acomodações, de acordo com o tipo e o número de listagens em que aparecem que são apresentadas na Figura 4.5.

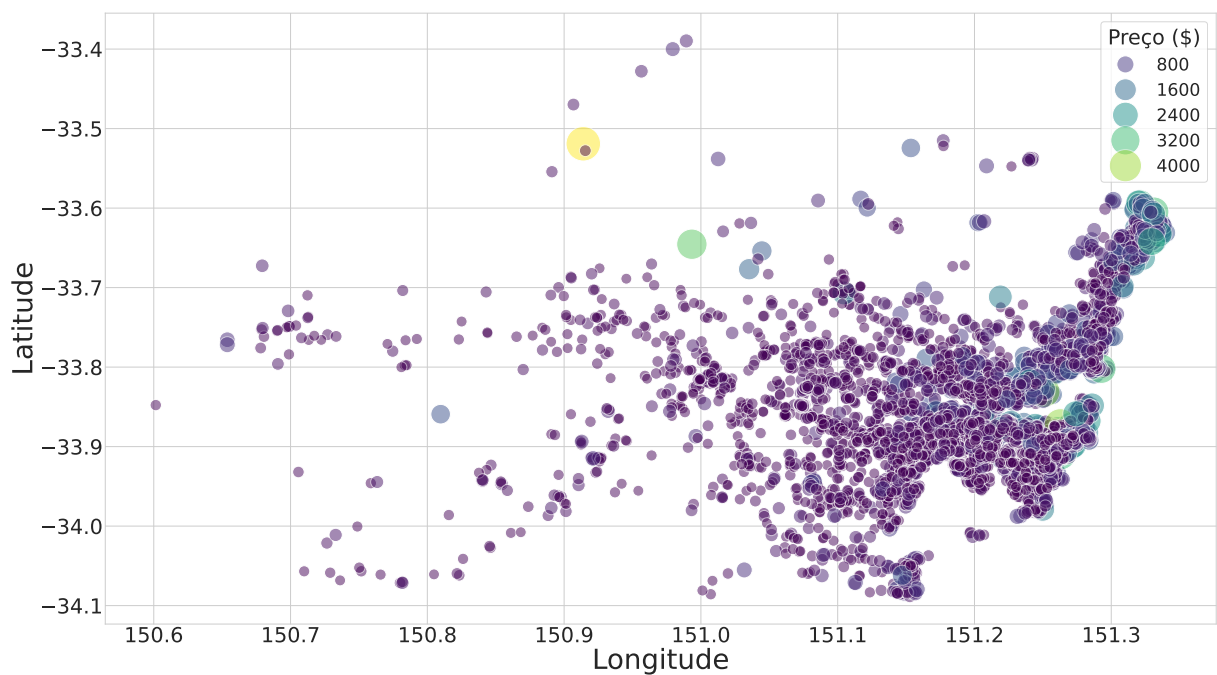


Figura 4.5 – Ranking de comodidades: Top 20



No que tange à análise geoespacial, foi gerado um mapa de calor, onde as áreas mais caras receberam tons mais claros, e as mais baratas, tons mais escuros, e o tamanho do ponto do mapa é proporcional ao preço. O mapa é ilustrado na Figura 4.6.

Figura 4.6 – Distribuição geográfica dos imóveis por preço



Em seguida, foi criado um novo mapa de calor interativo para melhor visualização da disposição dos preços dos imóveis. Áreas com muitos preços caros ficam mais quentes, e vice-versa. O centro inicial do mapa foi definido a partir das medianas da latitude e longitude, de forma que o mesmo seja iniciado no ponto mais denso da distribuição.

É possível identificar no mapa de calor interativo que a maior concentração de preços elevados (evidenciada pelos tons mais quentes) está em regiões litorâneas e próximas ao centro da capital. Isso reforça a hipótese de que os imóveis localizados em áreas privilegiadas possuem um valor mais elevado no que tange ao seu preço e que, portanto, as coordenadas geográficas (latitude e longitude) são um fator determinante para a precificação dos mesmos.

Em seguida, os atributos com dados faltantes foram identificadas. O resultado deste processo foram duas listas distintas de atributos com valores ausentes: uma contendo os atributos numéricos e outra com os atributos categóricos, em preparação para a imputação específica. As estratégias de imputação escolhidas para cada tipo de atributo serão elucidadas a seguir.

Para todas os atributos numéricos com valores ausentes, a estratégia selecionada foi a imputação com a mediana, devido à alta assimetria dos dados para esses atributos. Para os atributos categóricos com valores ausentes, a estratégia adotada para a imputação foi a moda dos valores. Os atributos com formato de data, *first\_review* e *last\_review*, também foram imputados utilizando a moda. a Moda foi utilizada por esses atributos não representarem dados numéricos, o que descarta o uso da média ou mediana.

Em seguida, conforme o que foi proposto em etapas anteriores, foi realizado um processo de engenharia de atributos, para criar novos atributos com valor preditivo sobre o atributo de saída. Primeiramente, com base no atributo *amenities*, foi criado o atributo numérico *amenities\_count*, que calcula o número total de comodidades oferecidas pela acomodação. Em sequência, como forma de agregar potencial preditivo em comodidades exclusivas, foi criada a atributo binário *has\_luxury\_amenities*, que indica se o imóvel possui alguma comodidade de luxo.

Prosseguiu-se com um processo de codificação do atributo categórico referente ao nome do bairro onde a acomodação se encontra, através da estratégia conhecida como conhecida como Target Encoding, conforme apresentado por [Pargent et al. \(2022\)](#). Foi calculada a média de preço de cada bairro, e o nome de cada bairro foi substituído por esse valor, gerando o atributo

*target\_encoded\_location*. Devido a isso, foi possível capturar o valor financeiro associado à localização do imóvel.

Por fim, criou-se um atributo de interação binária, capaz de capturar que espaços maiores em um bairro mais valorizado potencializam o valor cobrado pela estadia. Desta forma, 4 novos atributos foram criados, resultando em 71 atributos no conjunto final.

A etapa seguinte foi a detecção e tratamento de anomalias nos atributos numéricos. Para isso, foi utilizada a técnica do intervalo interquartil (IQR), conforme proposto por [Montgomery \(2024\)](#). Assim, aplicou-se a winsorização dos dados, tratando 41 atributos numéricos.

Em seguida, foi realizada a codificação de atributos. Como os atributos categóricos identificados no conjunto de dados são, em sua maioria, de alta cardinalidade, e os algoritmos escolhidos são baseados em pontos de corte para a construção das árvores de decisão - o que reduz a possibilidade de os algoritmos compreenderem de maneira errônea uma relação ordinal, uma vez que eles se preocupam apenas com o valor numérico do corte - o método escolhido foi o da codificação de rótulos (Label Encoding). Com isso, o número de atributos foi mantido em 71, sendo agora todas numéricas.

A próxima etapa do pré-processamento de dados foi a análise de multicolinearidade dos atributos de entrada. A primeira técnica utilizada, já abordada anteriormente, foi o coeficiente de correlação de Pearson. Sendo assim, para cada par de atributos de entrada, o atributo com mais de uma identificação de colinearidade (o que indicaria presença de multicolinearidade) foi removido do conjunto final. Esse processo resultou na remoção de 10 atributos de entrada, totalizando 60 atributos.

Em sequência, para confirmar o tratamento da multicolinearidade, foi calculado o VIF para os atributos de entrada. Caso o resultado fosse maior ou igual a 10, o atributo seria removido do conjunto final. O resultado foi a não remoção de nenhuma coluna, uma vez que todos os atributos com indícios de multicolinearidade já haviam sido removidos anteriormente.

O processo final do pré-processamento de dados foi divisão do conjunto de dados usando a validação cruzada  $K$ -Fold para 5 folds. Nesse processo, o conjunto principal foi dividido aleatoriamente em 10 subconjuntos disjuntos e iguais, sendo 5 de treino e 5 de teste. Assim, foi finalizado o processo de preparação dos dados para serem utilizados no treinamento dos modelos.

Sequencialmente, foi realizado o processo de busca em grade, utilizando a validação cruzada, para os algoritmos, da seguinte forma: a busca em grade selecionou a combinação de hiperparâmetros, executou o treinamento e teste do algoritmo e mediu o  $R^2$  de cada fold. Esse processo foi realizado de maneira iterativa para todos os folds e para todas as combinações possíveis, de forma que, ao final de cada iteração, a busca em grade mede o valor médio do  $R^2$  da combinação entre os 5 folds. Assim, ao final, as médias dessa métrica são comparadas entre as diferentes combinações e a melhor combinação é determinada com o maior valor médio encontrado. Os resultados para cada algoritmo estão elucidados na Tabela 4.4.

Tabela 4.4 – Resultados do Coeficiente de Determinação do Grid Search

| Fold  | LGBM   | RF     | XGB    | Média  | D.P.   |
|-------|--------|--------|--------|--------|--------|
| 1     | 0.9322 | 0.9990 | 0.9957 | 0.9756 | 0.0376 |
| 2     | 0.9366 | 0.9989 | 0.9957 | 0.9770 | 0.0350 |
| 3     | 0.9311 | 0.9990 | 0.9956 | 0.9752 | 0.0382 |
| 4     | 0.9373 | 0.9990 | 0.9953 | 0.9772 | 0.0346 |
| 5     | 0.9363 | 0.9991 | 0.9957 | 0.9770 | 0.0353 |
| Média | 0.9347 | 0.9990 | 0.9956 | —      |        |
| D.P.  | 0.0025 | 0.0001 | 0.0001 |        |        |

Em seguida, foram realizados o treino e teste (avaliação) de cada modelo. Os resultados na Tabela 4.5 demonstram que o LightGBM foi superior para o teste em todos os folds para cada uma das métricas de avaliação (apresentando os menores valores médios para o MAE, MSE e RMSE, e o maior valor médio para o  $R^2$ ), o que o tornou um forte candidato a ser a melhor escolha entre os três algoritmos. O tamanho do modelo em memória e o tempo decorrido para cada um dos processos, assim como para a busca em grade, também foram medidos, e serão apresentados em sequência.

Adicionalmente, foram registrados os tempos decorridos para a busca em grade, e avaliação de cada modelo, além do tamanho do modelo em memória para cada fold durante o treino e teste. Na Tabela 4.6, é possível perceber superioridade do LightGBM para o tempo de busca em grade e também de treino, enquanto o XGBoost foi ligeiramente superior no tempo de teste.

O LightGBM também foi superior quanto ao consumo de memória durante o treino e teste, apresentando a menor média entre os algoritmos.

Tabela 4.5 – Resultados de desempenho da avaliação dos modelos (D.P.: desvio padrão)

| Modelo | Fold | MAE     | MSE        | RMSE     | R <sup>2</sup> |
|--------|------|---------|------------|----------|----------------|
| LGBM   | 1    | 61.9776 | 20352.3258 | 142.6616 | 0.6880         |
|        | 2    | 60.9494 | 14458.1860 | 120.2422 | 0.7677         |
|        | 3    | 61.2023 | 15637.3976 | 125.0496 | 0.7693         |
|        | 4    | 61.0540 | 16333.4256 | 127.8023 | 0.7452         |
|        | 5    | 62.6282 | 17073.2301 | 130.6646 | 0.6887         |
| Média  |      | 61.5623 | 16770.9130 | 129.2840 | 0.7318         |
| D.P.   |      | 0.6438  | 1987.1020  | 7.5199   | 0.0365         |
| RF     | 1    | 67.0780 | 22131.1034 | 148.7653 | 0.6608         |
|        | 2    | 66.4024 | 16318.8839 | 127.7454 | 0.7379         |
|        | 3    | 65.3317 | 18606.1058 | 136.4042 | 0.7255         |
|        | 4    | 66.7374 | 17869.9162 | 133.6784 | 0.7212         |
|        | 5    | 68.2731 | 20697.2025 | 143.8652 | 0.6226         |
| Média  |      | 66.7645 | 19124.6424 | 138.0917 | 0.6936         |
| D.P.   |      | 0.9549  | 2060.4491  | 7.4381   | 0.0444         |
| XGB    | 1    | 65.0151 | 22201.0277 | 149.0001 | 0.6597         |
|        | 2    | 62.4023 | 15833.1370 | 125.8298 | 0.7457         |
|        | 3    | 60.1805 | 15782.2045 | 125.6272 | 0.7671         |
|        | 4    | 62.6780 | 16953.5325 | 130.2057 | 0.7355         |
|        | 5    | 64.0647 | 18460.9773 | 135.8712 | 0.6634         |
| Média  |      | 62.8681 | 17846.1758 | 133.3068 | 0.7143         |
| D.P.   |      | 1.6443  | 2385.4489  | 8.6874   | 0.0443         |

A utilização da abordagem baseada em aumento de gradiente (gradient boosting), tanto do LightGBM quanto do XGBoost, em comparação a estratégia gulosa adotada pelo Random Forest, no que diz respeito ao treinamento das árvores, foi determinante para a superioridade dos dois primeiros em relação ao último. O foco na correção sequencial do erro cometido pelas árvores foi de suma importância para que os algoritmos baseados na primeira estratégia fossem

Tabela 4.6 – Resultados de Tempo de Treino, Teste e Busca em Grade

| Modelo | Fold | Treino (s) | Teste (s) | Tamanho (MB) | Grid Search (min) |
|--------|------|------------|-----------|--------------|-------------------|
| LGBM   | 1    | 0.3804     | 0.0206    | 0.2636       | 10.49             |
|        | 2    | 0.3941     | 0.0210    | 0.2640       |                   |
|        | 3    | 0.3817     | 0.0204    | 0.2638       |                   |
|        | 4    | 0.4016     | 0.0214    | 0.2636       |                   |
|        | 5    | 0.3711     | 0.0219    | 0.2636       |                   |
| Média  |      | 0.3858     | 0.0210    | 0.2637       |                   |
| D.P.   |      | 0.0108     | 0.0005    | 0.0002       |                   |
| RF     | 1    | 18.1357    | 0.0761    | 57.2661      | 132.10            |
|        | 2    | 18.5543    | 0.1300    | 57.2340      |                   |
|        | 3    | 18.9058    | 0.0743    | 57.2562      |                   |
|        | 4    | 17.7537    | 0.0717    | 57.1185      |                   |
|        | 5    | 18.8670    | 0.2710    | 57.3568      |                   |
| Média  |      | 18.4433    | 0.1246    | 57.2463      |                   |
| D.P.   |      | 0.4418     | 0.0763    | 0.0764       |                   |
| XGB    | 1    | 5.4087     | 0.0261    | 2.1218       | 10.37             |
|        | 2    | 3.3432     | 0.0197    | 2.0900       |                   |
|        | 3    | 3.3778     | 0.0193    | 2.0638       |                   |
|        | 4    | 5.8360     | 0.0214    | 2.2508       |                   |
|        | 5    | 3.3348     | 0.0194    | 2.1095       |                   |
| Média  |      | 4.2601     | 0.0212    | 2.1272       |                   |
| D.P.   |      | 1.1205     | 0.0026    | 0.0648       |                   |

superiores ao seu concorrente que apresenta baixo foco na perda, por ser focado apenas na média dos resultados de cada árvore.

As otimizações apresentadas pelo LightGBM, como a construção das árvores “melhor primeiro” (contrária a abordagem utilizada pelo XGBoost e Random Forest), que potencializou a velocidade com o que o algoritmo construiu os resultados, o EFB (Exclusive Feature Bundling), que permitiu menor complexidade para lidar com muitos atributos, e o GOSS (Gradient-based One-Side Sampling), que auxiliou a reduzir o volume de dados ao mesmo tempo em que acelerava o treinamento sem perder muita precisão, foram fatores fundamentais para a superioridade do modelo em relação a seus concorrentes. É também possível afirmar que o desempenho

superior do LightGBM em relação aos seus concorrentes se da pela combinação entre o poder do aumento em gradiente e as otimizações apresentadas por ele, o que o tornam um modelo mais rápido, leve e preciso.

A fim de avaliar se a diferença entre os modelos foi estatisticamente significativa, foi realizado o teste de Friedman para comparar o desempenho de cada modelo para cada métrica de avaliação. Para cada métrica, o teste avaliou o desempenho dos modelos, de forma que para MAE, MSE e RMSE, o menor valor médio de desempenho ficou em primeiro lugar no ranking (rank 1) de Friedman, o segundo menor ficou em segundo lugar (rank 2) e o terceiro menor em terceiro lugar (rank 3). Para o  $R^2$ , o primeiro lugar foi o maior valor. Para cada métrica, o teste encontrou um  $p - valor$ , responsável por determinar se existia diferença significativa entre os modelos ( $p - valor < 0.05$ ) ou se a hipótese nula de que o desempenho dos três seria o mesmo era verdadeira. O teste revelou os seguintes resultados: para todas as métricas, foi encontrado o  $p - valor$  de 0.014996. Esse resultado rejeitou a hipótese nula e, portanto, identificou a existência de diferença significativa entre os modelos. Os rankings médios (entre todos os folds) calculados pelo teste foram:

- Para o MAE: LightGBM: 1.20, Random Forest: 3.00, XGBoost: 1.80;
- Para o MSE: LightGBM: 1.00, Random Forest: 2.80, XGBoost: 2.20;
- Para o RMSE: LightGBM: 1.00, Random Forest: 2.80, XGBoost: 2.20;
- Para o  $R^2$ : LightGBM: 1.00, Random Forest: 2.80, XGBoost: 2.20;

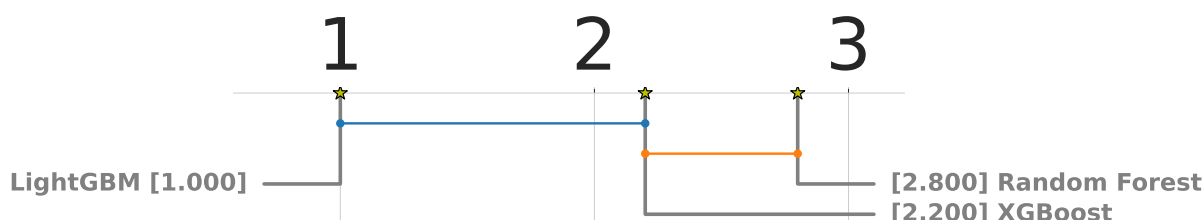
Os resultados confirmaram a superioridade do LightGBM, com o melhor rank médio entre as métricas (mais próximo de 1) quando comparado a seus concorrentes. É possível afirmar que o algoritmo foi o mais eficaz, obtendo o menor erro e o melhor ajuste entre os três. O modelo Random Forest obteve o pior desempenho, com os ranks médios mais altos (piores) entre os modelos. Além disso, devido ao resultado encontrado para o  $p - valor$ , que confirmou a existência de diferença significativa entre os modelos, surgiu a necessidade de se realizar o teste de Nemenyi.

Portanto, foi realizado o teste de Nemenyi, a fim de identificar algoritmos possuem diferença significativa entre eles. A diferença de desempenho entre os algoritmos foi considerada

significativa no momento em que a diferença entre os seus ranks médios correspondentes (calculados no teste de Friedman) fosse maior ou igual a Distância Crítica (que é igual a 1.483, calculada para: 3 modelos; 5 folds;  $\alpha = 0.05$ ; valor crítico do intervalo studentizado = 2.344). Os resultados do teste, ilustrado na Figura 4.7 para a métrica MAE, confirmam que existe uma diferença significativa entre o LightGBM e as Random Forests.

Percebe-se que, no diagrama de distância crítica, não existe uma conexão horizontal que liga os dois modelos diretamente, o que confirma visualmente que:  $1.00 \text{ (LightGBM)} - 2.8 \text{ (Random Forest)} = 1.8$ , que é maior que o valor da Distância Crítica, enquanto os outros pares de algoritmos (LightGBM vs. XGBoost; XGBoost vs. Random Forest) não apresentaram diferença significativa. Existe uma conexão horizontal que liga LightGBM ao Random XGBoost, e outra que liga o XGBoost ao Random Forest, que mostra que a diferença do rank médio entre eles é menor que a Distância Crítica. Essas conexões confirmam que existe uma semelhança de desempenho entre eles, isto é, a hipótese de que existe diferença significativa foi descartada. Para as outras métricas, as mesmas informações foram confirmadas.

Figura 4.7 – Diagrama de distância crítica

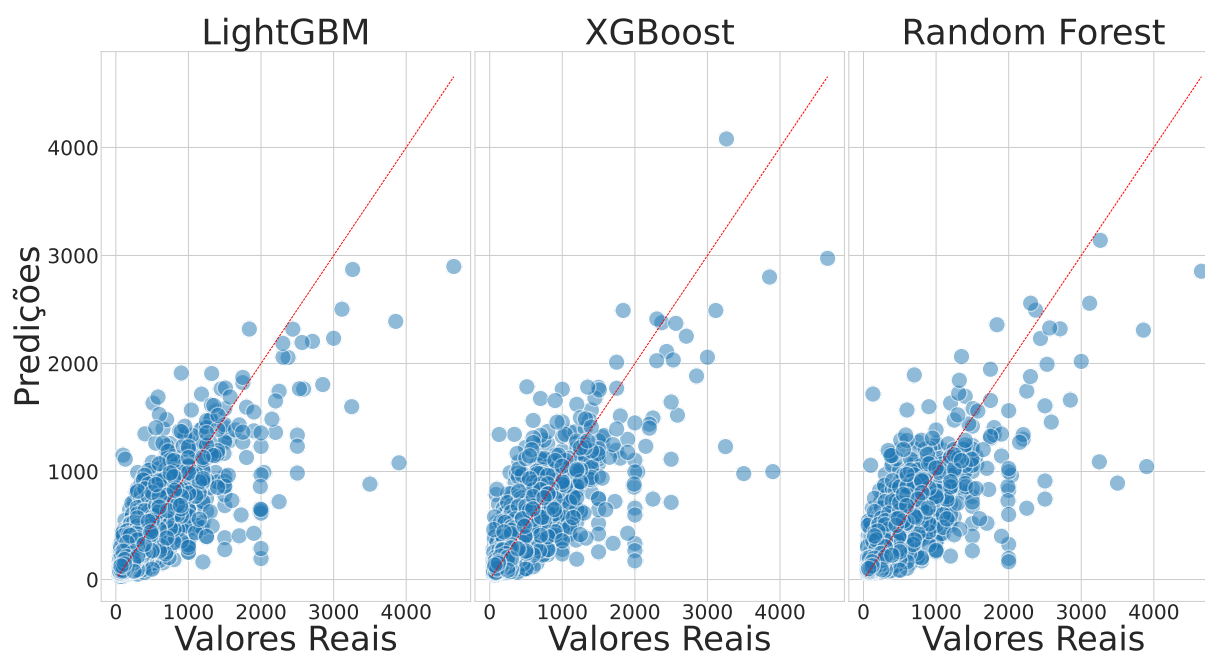


Por fim, foi realizada uma análise dos resíduos vs. valores observados (valores reais), a fim de compreender a confiabilidade dos modelos e validar a adequação deles para a predição dos valores desejados. Para isso, foram gerados os gráficos demonstrados na Figura 4.8. A partir desses gráficos foi possível perceber que os modelos apresentaram boa precisão quanto à predição dos valores. Quanto mais próximos os pontos azuis (coordenadas que representam o valor onde os eixos dos gráficos se encontram) da diagonal vermelha (linha de identidade, onde  $X = Y$ ), melhor foram os resultados dos resíduos. Assim, percebe-se satisfatória a proximidade dos pontos em relação à diagonal, especialmente para valores mais baixos, o que indicou boa confiabilidade dos algoritmos no que tange à predição do preço de acomodações mais baratas. Entretanto, foi compreendido que para todos os modelos, existiu certa incerteza na predição de



valores mais altos, ilustrada pela maior dispersão desses resíduos nos gráficos (mais distantes da diagonal, o que indica menor precisão). Concluiu-se, portanto, que os algoritmos desempenharam bem na predição de valores mais baixos, enquanto apresentaram maior dificuldade para prever valores mais elevados.

Figura 4.8 – Resíduos vs. Valores Observados

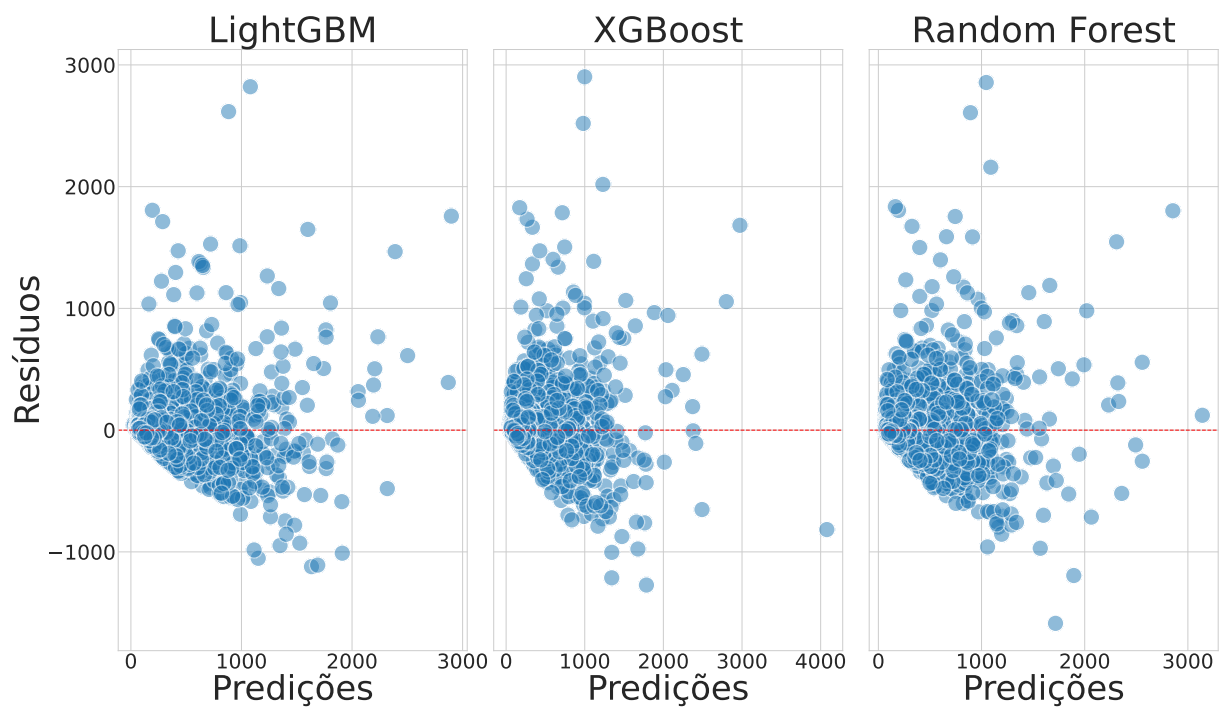


Em complemento, também foram gerados gráficos dos resíduos vs. predições, a fim de compreender se os modelos estão bem ajustados, mesmo que o resultado das métricas de avaliação tenha sido satisfatório. Assim, um modelo bem ajustado deveria apresentar uma nuvem de pontos aleatória e sem padrão, distribuída de forma simétrica em torno da linha horizontal do zero. Os gráficos, ilustrados na Figura 4.9, indicaram distribuição aleatória dos pontos em relação ao eixo Y, resultado satisfatório conforme o desejado.

Os gráficos revelaram que, embora a média dos pontos estivesse próxima do zero, existe uma concentração de pontos negativos (abaixo do zero, na parte das baixas predições), que indica que os modelos fizeram predições ligeiramente acima do valor observado para esse contexto. Além disso, existe uma concentração de pontos positivos (acima do zero, na parte das altas predições), que indicou que os algoritmos fizeram predições abaixo do valor real para esse contexto, especialmente para anomalias (valores muito altos).

Os gráficos também revelaram a presença de heterocedasticidade para todos os três modelos, especialmente no que diz respeito à predição de valores mais altos, indicando menor confiabilidade nesse cenário. Desta forma, seria necessário adotar medidas para tratar dessa questão, a fim de tornar os modelos mais confiáveis e generalizáveis, ainda que o desempenho tenha sido satisfatório para a maioria das observações.

Figura 4.9 – Resíduos vs. Predições



## 5 CONCLUSÃO

Os dados desse estudo indicam que o algoritmo de regressão que apresentou o melhor desempenho quanto a predição do preço de uma acomodação do Airbnb em Sydney e região foi o LightGBM. O algoritmo se demonstrou superior não só em todas as métricas de avaliação selecionadas para o projeto (MAE, MSE, RMSE e  $R^2$ ), mas também na eficiência computacional, consumindo menos memória e executando os processos de maneira mais rápida do que seus concorrentes. Além disso, foi possível demonstrar, através da criação de mapas de calor e de novos atributos que refletiram essa relação, que a localização de uma acomodação é fundamental para a precificação da mesma. Entretanto, foi identificada menor confiabilidade, para os três modelos, na predição de preços mais elevados. Portanto, aplicar técnicas que mitiguem essa problemática e testar outros modelos mais robustos a heterocedasticidade pode ser interessante para treinar modelos ainda mais confiáveis e generalizáveis.

Para trabalhos futuros, algumas direções serão exploradas: primeiramente, será utilizado um conjunto de dados maior, abrangendo um maior número de acomodações e uma extensão geoespacial maior, permitindo uma análise mais robusta e completa. Em seguida, pretende-se investigar outros modelos de regressão e compará-los com os já utilizados nesse estudo, a fim de encontrar resultados ainda mais satisfatórios,

## REFERÊNCIAS

- ABUZAID, A.; ALKRONZ, E. A comparative study on univariate outlier winsorization methods in data science context. **Italian Journal of Applied Statistics**, v. 36, p. 85–99, 04 2024.
- AIRBNB. **About Us**. 2025. <<https://news.airbnb.com/about-us/>>. Accessed on: 2025-09-05.
- AKALIN, O.; ALPTEKIN, G. Enhancing airbnb price predictions with location-based data: A case study of istanbul. In: . [S.l.: s.n.], 2024. p. 207–212.
- BDAIR, M. Enhancing machine learning workflows: A comprehensive study of machine learning pipelines. 03 2022.
- BELYADI, H.; HAGHIGHAT, A. **Machine Learning Guide for Oil and Gas Using Python: A Step-by-Step Breakdown with Data, Algorithms, Codes, and Applications**. Gulf Professional Publishing, 2021. ISBN 9780128219300. Disponível em: <<https://books.google.es/books?id=MjoEEAAQBAJ>>.
- BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001.
- CHEN, T.; GUESTRIN, C. Xgboost: A scalable tree boosting system. **CoRR**, abs/1603.02754, 2016. Disponível em: <<http://arxiv.org/abs/1603.02754>>.
- CHICCO, D.; WARRENS, M. J.; JURMAN, G. The coefficient of determination r-squared is more informative than smape, mae, mape, mse and rmse in regression analysis evaluation. **PeerJ Computer Science**, v. 7, 2021. Disponível em: <<https://api.semanticscholar.org/CorpusID:236196832>>.
- COWLES, M.; DAVIS, C. On the origins of the .05 level of statistical significance. **American Psychologist**, v. 37, n. 5, p. 553–558, 1982.
- DAS, K. A brief review of tests for normality. **American Journal of Theoretical and Applied Statistics**, v. 5, p. 5, 01 2016.
- DEMIRTRK, D.; MİNTEMUR, O.; ARSLAN, A. Optimizing LightGBM and XGBoost algorithms for estimating compressive strength in high-performance concrete. **Arabian Journal for Science and Engineering**, 2025.
- DUDÁŠ, A. Graphical representation of data prediction potential: correlation graphs and correlation chains. **The Visual Computer**, Springer Nature, v. 40, p. 6969–6982, 2024.
- FORTHOFER, R. N.; LEE, E. S.; HERNANDEZ, M. 3 - descriptive methods. In: FORTHOFER, R. N.; LEE, E. S.; HERNANDEZ, M. (Ed.). **Biostatistics (Second Edition)**. Second edition. San Diego: Academic Press, 2007. p. 21–69. ISBN 978-0-12-369492-8. Disponível em: <<https://www.sciencedirect.com/science/article/pii/B978012369492850008X>>.

FREUND, Y.; SCHAPIRE, R. E. Experiments with a new boosting algorithm. In: **Proceedings of the Thirteenth International Conference on Machine Learning (ICML)**. Bari, Italy: Morgan Kaufmann, 1996. p. 148–156.

FRIEDMAN, J. H. **Greedy Function Approximation: A Gradient Boosting Machine**. Stanford, CA, 2001. IMS 1999 Reitz Lecture; modificado em Março de 2000 e Abril de 2001.

FRIEDMAN, M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. **Journal of the American Statistical Association**, ASA Website, v. 32, n. 200, p. 675–701, 1937.

GALTON, F. Regression towards mediocrity in hereditary stature. **The Journal of the Anthropological Institute of Great Britain and Ireland**, [Royal Anthropological Institute of Great Britain and Ireland, Wiley], v. 15, p. 246–263, 1886. ISSN 09595295, 23972564. Disponível em: <<http://www.jstor.org/stable/2841583>>.

GOOGLE. 2025. <<https://research.google.com/colaboratory/intl/pt-BR/faq.html#whats-colaboratory>>. Accessed on: 2025-11-07.

GROVE, C. C.; FISHER, R. A. Statistical methods for research workers. **Nature**, v. 131, p. 383–383, 1930. Disponível em: <<https://api.semanticscholar.org/CorpusID:4073304>>.

HAMEED, W.; ALI, N. Missing value imputation techniques: A survey. **UHD Journal of Science and Technology**, v. 7, p. 72–81, 03 2023.

HUANG, Y. The sharing economy puzzle an overview case of airbnb. **Advances in Economics, Management and Political Sciences**, v. 195, p. 180–185, 07 2025.

KAGGLE. 2025. <<https://www.kaggle.com/>>. Accessed on: 2025-11-07.

KE, G. et al. LightGBM: A highly efficient gradient boosting decision tree. In: **Advances in Neural Information Processing Systems 30**. Long Beach, CA, USA: [s.n.], 2017. p. 3146–3154.

KERN, C.; KLAUSCH, T.; KREUTER, F. Tree-based machine learning methods for survey research. **Survey Research Methods**, v. 13, p. 73–93, 04 2019.

LIASHCHYNSKYI, P.; LIASHCHYNSKYI, P. **Grid Search, Random Search, Genetic Algorithm: A Big Comparison for NAS**. 2019.

MONTGOMERY, R. **Techniques for Outlier Detection A Comprehensive View**. 2024.

NEMENYI, P. **Distribution-free Multiple Comparisons**. Princeton University, 1963. Disponível em: <<https://books.google.com.br/books?id=nhDMtgAACAAJ>>.

OLUSEGUN, J. Handling categorical variables in ensemble algorithms. 02 2025.

PARGENT, F. et al. Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features. **Computational Statistics**, v. 37, p. 1–22, 03 2022.

PEARSON, K. Determination of the coefficient of correlation. **Science**, American Association for the Advancement of Science, v. 30, n. 757, p. 23–25, 1909. ISSN 00368075, 10959203. Disponível em: <<http://www.jstor.org/stable/1635783>>.

PLEVRIS, V. et al. Investigation of performance metrics in regression analysis and machine learning-based prediction models. In: . [S.l.: s.n.], 2022.

RAINIO, O.; TEUHO, J.; KLÉN, R. Evaluation metrics and statistical tests for machine learning. **Scientific Reports**, v. 14, 03 2024.

REZAZADEH, S. Review of machine learning. **TMP Universal Journal of Research and Review Archives**, v. 4, 07 2025.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. **IBM Journal of Research and Development**, v. 3, n. 3, p. 210–229, July 1959.

SANTHANAM, R. et al. Experimenting xgboost algorithm for prediction and classification of different datasets. In: . [S.l.: s.n.], 2017.

SHAPIRO, S. S.; WILK, M. B. An analysis of variance test for normality (complete samples). **Biometrika**, v. 52, n. 3/4, p. 591–611, 1965. Disponível em: <<http://links.jstor.org/sici?sici=0006-3444%28196512%2952%3A3%2F4%3C591%3AAAQVTF%3E2.0.CO%3B2-B>>.

SHRESTHA, N. Detecting multicollinearity in regression analysis. **American Journal of Applied Mathematics and Statistics**, v. 8, p. 39–42, 06 2020.

THAI, H.-T. Machine learning for structural engineering: A state-of-the-art review. **Structures**, v. 38, p. 448–491, 04 2022.

VERMA, V. A comprehensive framework for residual analysis in regression and machine learning. **Journal of Information Systems Engineering and Management**, v. 10, p. 1–18, 01 2025.

WONG, T.-T. Performance evaluation of classification algorithms by k-fold and leave-one-out cross validation. **Pattern Recognition**, v. 48, 09 2015.

ZENG, G.; HUO, N.; ZHANG, X. Research on different factors affecting airbnb housing prices. **Advances in Economics, Management and Political Sciences**, v. 202, p. 37–49, 07 2025.